

FIAP POS TECH
ARQUITETURA CLOUD E DEVOPS

Hackathon SolidaryTech

Plataforma de Doacoes para ONGs — Documento Tecnico Completo

Setembro 2026

EQUIPE

Nome	RM	Email
Diego Felipe Rocha Silva	rm369588	diegofeliperochasilva@gmail.com
Erick Saraiva de Sousa	rm369969	ericksaraiva27@gmail.com
Jhousyfran Muniz Costa	rm369476	jhousyfrancosta@gmail.com
Regis Teruo Nomi	rm369601	regis.nomi@gmail.com
Yuri Jose do Carmo	rm370037	yjcarmo@gmail.com

Sumario

1. 1. Pitch Executivo

2. 2. Evolucao do Projeto

3. 3. Decisoos Tecnicas

4. 4. Plano de Continuidade de Negocios

5. 5. SRE - SLO, SLI e SLA

6. 6. ITSM e AIOps

7. 7. FinOps e Forecast

8. 8. README do Projeto

SolidaryTech - Pitch Executivo

Conectando Generosidade a Quem Mais Precisa

O Problema

O Brasil possui mais de **815 mil organizacoes da sociedade civil** (IPEA, 2023). A maioria enfrenta desafios criticos:

- 72%** das ONGs nao possuem infraestrutura digital confiavel para receber doacoes
- Plataformas tradicionais cobram **taxas de ate 15%** sobre cada doacao
- Quando uma campanha viraliza, os sistemas **caem nos momentos de maior engajamento**, desperdicando o pico de generosidade
- Nao ha **rastreabilidade** do impacto: doadores nao sabem se sua contribuicao chegou ao destino
- O matching entre **voluntarios e campanhas** e feito manualmente, com planilhas

Resultado: Milhoes de reais em doacoes perdidas e milhares de voluntarios sem direcionamento.

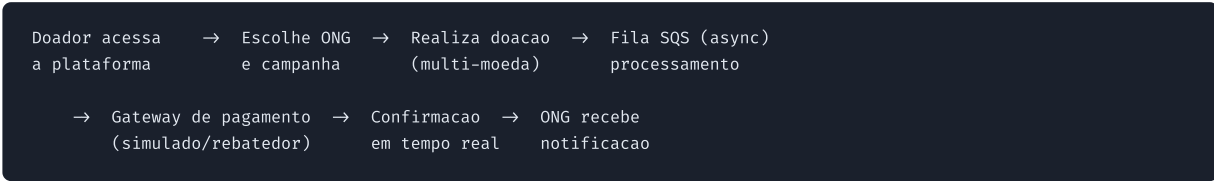
A Solucao: SolidaryTech

A **SolidaryTech** e uma plataforma cloud-native de codigo aberto que conecta **ONGs, doadores e voluntarios** em um ecossistema integrado, confiavel e global.

3 Microservicos, 1 Missao

Servico	Funcao	Tecnologia
ngo-service	Cadastro e gestao de ONGs parceiras	Python/Flask + PostgreSQL
donation-service	Processamento de doacoes (Hot Path)	Python/Flask + SQS + PostgreSQL
volunteer-service	Matching inteligente entre voluntarios e campanhas	Python/Flask + PostgreSQL

Fluxo de uma Doacao



Diferenciais Competitivos

1. Disponibilidade de 99.9% (SLA)

A plataforma foi construida sobre principios de **Site Reliability Engineering (SRE)**:

- SLI de Latencia:** 99.9% das transacoes processadas em menos de 500ms
- SLI de Disponibilidade:** 99.9% de uptime (apenas ~43 min de indisponibilidade por mes)
- Error Budget:** politica formal de consumo com alertas automaticos
- Self-healing:** pods com falha sao automaticamente substituidos pelo Kubernetes

2. Doacoes Multi-Moeda

A SolidaryTech e uma **plataforma global**. Aceitamos doacoes em:

Moeda	Simbolo	Regiao
Real Brasileiro	BRL	Brasil
Dolar Americano	USD	Americas
Euro	EUR	Europa
Libra Esterlina	GBP	Reino Unido
Iene Japones	JPY	Asia

Todas as transacoes sao convertidas automaticamente e registradas com rastreabilidade completa.

3. Disaster Recovery com Integridade de Dados

Se a nuvem cair, as doacoes **continuam**:

- **RTO**: 3-5 minutos para restaurar o servico critico de doacoes (em teste de lab: 58 segundos)
- **RPO = 0 para dados commitados**: sync bidirecional (`dr-data-sync.sh`) preserva todos os dados persistidos
- **Analise honesta**: ~4 transacoes in-flight perdidas durante a janela de failover (~R\$ 7.100 de impacto)
- **Warm Standby** sempre ativo em us-west-2 (1 node EKS + RDS + apps com 1 replica)
- **Failback transparente**: `dr-failback.sh` com dual-active transitorio — zero downtime para o cliente na volta
- **Identificacao de regioao**: panel "Regiao Ativa" em todos os 6 dashboards Grafana + endpoint `/region` nos microservicos
- Health checker automatico (`dr-health-checker.sh --daemon --auto-failover`) dispara failover apos 3 falhas consecutivas

4. Custos Otimizados com FinOps

Cada centavo importa quando se trata de uma ONG:

Recurso	Custo Mensal
Infraestrutura completa (EKS + RDS + SQS + S3)	~\$200/mes
DR Warm Standby	~\$152/mes
Total	~\$352/mes

Com otimizacoes aplicaveis (Spot Instances, Reserved, scale-down): - **Economia projetada**: ate 25% (~\$88/mes) - **Custo otimizado**: ~\$264/mes para uma plataforma enterprise-grade

5. Monitoramento Preditivo (AIOps)

Nao esperamos o problema acontecer - **prevemos**:

- Deteccao de anomalias via Prometheus + regras de ML
- Alertas proativos quando o Error Budget esta sendo consumido rapidamente
- Root Cause Analysis (RCA) automatizado
- Post-Mortem blameless com acoes automaticas

6. Seguranca End-to-End (Hardening Completo)

- Dados criptografados em transito (TLS) e em repouso (AES-256)
- Banco de dados acessivel **somente pela rede interna** (private subnets) com **deletion protection** ativo
- Security Groups com regra de menor privilegio
- Scans automaticos de vulnerabilidades (Trivy SAST/SCA) em cada deploy
- **Containers hardened**: securityContext non-root, readOnlyRootFilesystem, capabilities drop ALL, automountServiceAccountToken desabilitado
- **NetworkPolicies**: deny-all default + regras explicitas por namespace (incluindo monitoring)

- **ECR imutavel:** tags de imagem IMMUTABLE para prevenir supply chain attacks
- **Credenciais protegidas:** mascaramento em logs, variaveis de ambiente (sem exposicao via `ps aux`), `chmod 600` em arquivos de credenciais
- **Validacao de entrada:** paginacao limitada (max 100), validacao de valores monetarios (rejeita NaN/Inf/negativos)
- **EKS:** public_access_cidrs configuravel, audit logs habilitados
- **LGPD-ready:** dados de doadores protegidos conforme legislacao brasileira

Arquitetura de Alta Disponibilidade



ROI - Retorno sobre Investimento

Comparacao com Solucoes Tradicionais

Aspecto	Solucao Tradicional	SolidaryTech
Custo mensal de infraestrutura	\$2.000 - \$5.000	\$352
Taxas por transacao	5% - 15%	0% (open-source)
Tempo de recuperacao (desastre)	4-24 horas	3-5 minutos (teste: 58s)
Monitoramento	Manual/reactivo	Automatico/preditivo
Escalabilidade	Semanas para provisionar	Minutos (auto-scaling)
Vendor lock-in	Alto	Nenhum (OSS + multi-cloud ready)

Cenario: ONG com 10.000 doacoes/mes (ticket medio R\$50)

- **Volume mensal:** R\$500.000 em doacoes
- **Economia em taxas** (vs plataforma com 10%): **R\$50.000/mes**
- **Custo da infraestrutura:** ~R\$1.800/mes
- **ROI liquido mensal:** **R\$48.200**

Escalabilidade

A plataforma foi projetada para escalar automaticamente:

- **HPA (Horizontal Pod Autoscaler):** donation-service (Hot Path) escala de 1 a 8 replicas; demais servicos de 1 a 4 replicas
- **Cluster Autoscaler:** adiciona nodes ao EKS quando necessario (1 a 5 nodes)
- **SQS:** filas decoupled absorvem picos sem impactar o usuario
- **Testado para:** campanhas virais com 10x o trafego normal

Cenario: Black Friday Solidaria

Uma campanha viraliza no Instagram. O trafego sobe 10x em 30 minutos:

1. HPA detecta aumento de CPU (>60% no donation-service) e cria replicas adicionais (ate 4 pods/30s)
2. Cluster Autoscaler provisiona novos nodes (t3.medium)
3. SQS absorve o burst de doacoes, processando de forma assincrona
4. Doadores nao percebem nenhuma lentidao
5. Apos o pico, o sistema escala de volta automaticamente (economia de custos)

Modelo de Sustentabilidade

A SolidaryTech e uma iniciativa sem fins lucrativos:

- **Zero taxas** sobre doacoes - 100% do valor chega as ONGs
- **Codigo aberto** - qualquer organizacao pode auditar e contribuir
- **Custos sustentados** por:
 - Parcerias com empresas de tecnologia (AWS Credits for Nonprofits)
 - Grants de fundacoes internacionais
 - Programa de voluntariado corporativo (empresas cedem engenheiros)

Conformidade e Governanca

Requisito	Status
LGPD (Lei Geral de Protecao de Dados)	Implementado
Criptografia em transito e repouso	Implementado
Auditoria de acessos	Implementado (CloudTrail)
Backup e retencao de dados	7 dias automatico + DR
Segregacao de ambientes	Produccao / DR separados
Gestao de segredos	Kubernetes Secrets + rotation

Equipe

Consulte o documento de entrega do projeto (PDF) para a lista completa de integrantes.

Conclusao

A SolidaryTech nao e apenas uma plataforma de tecnologia - e um **acelerador de impacto social**. Com uma infraestrutura que custa menos que um aluguel de escritorio, conseguimos:

- Garantir que **100% das doacoes** cheguem as ONGs
- Operar com **99.9% de disponibilidade** mesmo em momentos criticos
- Recuperar de desastres em **menos de 5 minutos** com integridade de dados (teste em lab: 58s)
- Escalar para atender **qualquer volume** de generosidade

Cada linha de codigo, cada regra de monitoramento, cada decisao arquitetural foi feita pensando em uma coisa: que nenhuma doacao se perca.

"A tecnologia a servico de quem mais precisa."

Evolucao do Projeto - Fases 1 a 5

SolidaryTech: Da Fundacao a Maturidade Operacional

Timeline do Projeto

Fase 1	Fase 2	Fase 3	Fase 4	Fase 5
Fundacao	Containers	Automacao	Observabilidade	Maturidade
v	v	v	v	v
[Docker]	[K8s Avancado]	[Terraform]	[Prometheus]	[SRE/FinOps]
[K8s Basics]	[Microservicos]	[CI/CD]	[Grafana]	[ITSM/AIOps]
[AWS Basics]	[Networking]	[DevSecOps]	[APM/Tracing]	[DR/PCN]
[Arquitetura]	[Storage]	[GitOps]	[Alertas]	[Seguranca]

Fase 1 - Fundacao (Docker, Kubernetes e AWS)

O que foi aprendido

- Conceitos fundamentais de **containers**: imagens, layers, Dockerfile, Docker Hub
- Introducao ao **Kubernetes**: Pods, Deployments, Services, Namespaces
- Fundamentos de **arquitetura cloud**: regioes, AZs, VPC, subnets
- Servicos AWS basicos: EC2, S3, IAM, VPC
- Cultura **DevOps**: colaboracao, automacao, feedback continuo

O que foi aplicado no projeto

Tecnologia	Aplicacao na SolidaryTech
Docker	Dockerfiles multi-stage para os 3 microservicos
Kubernetes	Deployments com 2 replicas, Services ClusterIP
AWS VPC	Rede isolada com subnets publicas e privadas
AWS EKS	Cluster gerenciado Kubernetes 1.29
Namespaces	<code>solidarytech</code> (apps) e <code>monitoring</code> (observabilidade)

Diferencial entregue

Capacidade de empacotar qualquer aplicacao em container e orquestra-la em um cluster Kubernetes gerenciado na AWS.

Fase 2 - Containerizacao Avancada

O que foi aprendido

- Dockerfiles **otimizados**: multi-stage builds, cache de layers, imagens slim
- Orquestracao K8s avancada: **HPA**, liveness/readiness probes, resource limits

- Networking Kubernetes: Ingress, NetworkPolicies, DNS interno
- Persistencia: Volumes, PVCs, StatefulSets
- Padrao de **microsservicos**: decomposicao, comunicacao entre servicos

O que foi aplicado no projeto

Tecnologia	Aplicacao na SolidaryTech
Multi-stage builds	Imagens finais ~80MB (python:3.13-slim)
Non-root user	Seguranca: containers rodam sem privilegio root
HPA	Auto-scaling: donation-service 1-8 replicas (CPU 60%), demais 1-4 (CPU 70%)
Probes	Liveness (/health) e Readiness (/ready) em todos os servicos
Resource limits	CPU e memoria calibrados por servico (rightsizing)
Ingress	Roteamento por path para cada microsservico
Pod Anti-Affinity	Pods distribuidos em nodes diferentes para HA

Diferencial entregue

Microsservicos otimizados com auto-scaling inteligente, alta disponibilidade e uso eficiente de recursos.

Fase 3 - Automacao (IaC, CI/CD, DevSecOps, GitOps)

O que foi aprendido

- **Terraform**: modules, state, workspaces, remote backend
- **CI/CD**: GitHub Actions, pipelines multi-stage, gates de qualidade
- **DevSecOps**: SAST, SCA, scans de imagem, seguranca no pipeline
- **GitOps**: ArgoCD, FluxCD, reconciliacao automatica
- **Seguranca na Cloud**: IAM, Security Groups, encryption at rest

O que foi aplicado no projeto

Tecnologia	Aplicacao na SolidaryTech
Terraform	8 modulos (VPC, EKS, RDS, SQS, S3, ECR, DynamoDB, ElastiCache), 2 ambientes (prod, DR)
GitHub Actions	5 pipelines (3 servicos + seguranca + terraform)
Trivy	Scan SAST/SCA em cada build - bloqueia HIGH/CRITICAL
ArgoCD	App of Apps com auto-sync e self-heal
Remote State	S3 + DynamoDB para state locking
GitOps Flow	Push no main → CI build → Atualiza manifesto → ArgoCD sync

Pipeline CI/CD Completa



```
[Security Scan] → Trivy (imagem) + Safety (deps) + Checkov (IaC)
|
v
[Build + Push ECR]
|
v
[Update Manifests] → Altera tag da imagem no YAML
|
v
[ArgoCD Sync] → Detecta mudanca e aplica no cluster
|
v
[Producao!]
```

Diferencial entregue

Infraestrutura 100% como codigo, pipeline com seguranca integrada e entrega continua automatizada via GitOps.

Fase 4 - Observabilidade (Monitoring, APM, Tracing)

O que foi aprendido

- Stack de observabilidade: **Prometheus** (metricas), **Grafana** (visualizacao), **Loki** (logs)
- APM** e distributed tracing com OpenTelemetry
- Alertas e notificacoes: regras Prometheus, Alertmanager
- Instrumentacao de codigo: metricas customizadas, spans, trace propagation
- Golden Signals**: Latencia, Trafego, Erros, Saturacao

O que foi aplicado no projeto

Tecnologia	Aplicacao na SolidaryTech
Prometheus	Scraping de metricas de todos os servicos + K8s, <code>external_labels</code> para identificacao de regioao
Grafana	6 dashboards: Platform Overview, Business Metrics, SRE Golden Metrics, SLO Tracking, Infrastructure & Cluster, DR Status — todos com panel "Regiao Ativa"
Loki	Agregacao centralizada de logs
OpenTelemetry	Collector como DaemonSet + instrumentacao Flask
Distributed Tracing	Propagacao de contexto entre donation-service e worker
Metricas SLI	donation_request_duration_seconds, donation_errors_total
ServiceMonitor	CRD Prometheus para auto-discovery de endpoints

Metricas Customizadas do donation-service

```
DONATION_DURATION = Histogram("donation_request_duration_seconds", ...)
DONATION_ERRORS = Counter("donation_errors_total", ...)
DONATION_PROCESSED = Counter("donation_processed_total", ...)
DONATION_SLO_LATENCY = Histogram("donation_slo_latency_seconds", ...)
```

Diferencial entregue

Visibilidade completa do ecossistema com metricas, logs e traces correlacionados - nenhum "voo cego".

Fase 5 - Maturidade Operacional (SRE, FinOps, ITSM/AIOps, DR)

O que foi aprendido

- **SRE:** SLI, SLO, SLA, Error Budget, Runbooks, Post-Mortem
- **FinOps:** Tagging, Rightsizing, Forecast, otimizacao de custos
- **ITSM:** ITIL, gestao de incidentes, ciclo de vida, classificacao SEV1-4
- **AIOps:** Deteccao de anomalias, self-healing, RCA automatizado
- **DR:** PCN, RTO, RPO, estrategias (Active-Active vs Warm Standby)
- **Multicloud e Seguranca:** cross-region, NetworkPolicies, security contexts

O que foi aplicado no projeto

Tecnologia	Aplicacao na SolidaryTech
SLI/SLO	2 SLIs formais para donation-service (latencia + disponibilidade)
Error Budget	0.1% = 43.2 min/mes, com politica de consumo
Tagging FinOps	Todos os recursos com Project, Environment, CostCenter, ManagedBy
Rightsizing	Requests/limits calibrados + HPA
Forecast	Projecao de \$352/mes com 5 recomendacoes de otimizacao
ITSM	Ciclo completo: Deteccao → Classificacao → Escalonamento → Resolucao → Post-Mortem
AIOps	Alertas preditivos + self-healing K8s + RCA automatizado
DR	Warm Standby em us-west-2, deploy unificado, RTO 3-5 min (teste: 58s), sync bidirecional (<code>dr-data-sync.sh</code>), failback transparente (<code>dr-failback.sh</code>), identificacao de regioao em 3 camadas
PCN	Documento executivo com RTO 3-5 min (teste: 58s), RPO=0 para dados commitados, analise honesta de perda (~4 transacoes/~R\$7.100 por failover)
Seguranca	Private subnets, SecurityContext hardened (non-root, readOnly, drop ALL), NetworkPolicies deny-all, ECR IMMUTABLE, deletion_protection, automountServiceAccountToken:false

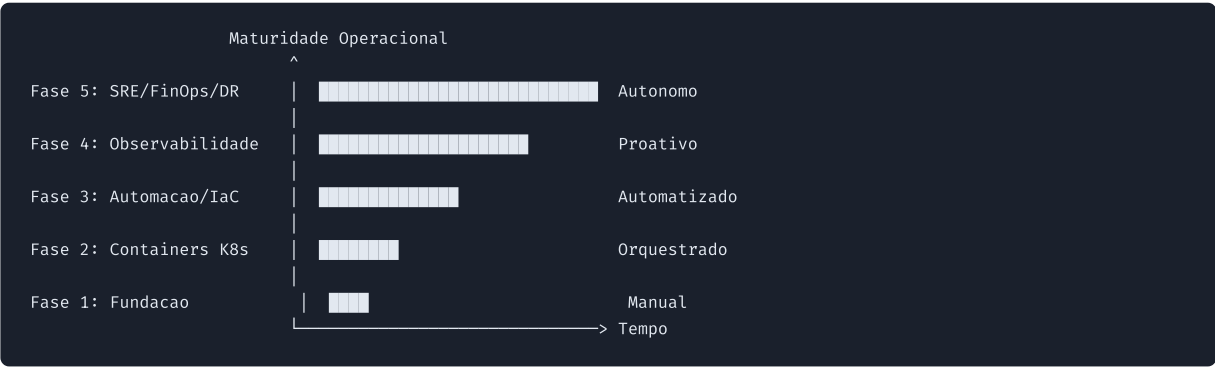
Diferencial entregue

Maturidade operacional enterprise: a plataforma nao apenas funciona - ela se auto-monitora, se auto-recupera, justifica seus custos e sobrevive a desastres.

Quadro Consolidado de Tecnologias

Categoria	Tecnologia	Fase Introduzida	Onde Aparece no Projeto
Container	Docker	Fase 1	microservices/*/Dockerfile
Orquestracao	Kubernetes (EKS)	Fase 1	kubernetes/base/
Cloud	AWS (VPC, EC2, RDS, SQS, S3)	Fase 1	terraform/modules/
IaC	Terraform	Fase 3	terraform/ (8 modulos, 2 ambientes)
CI/CD	GitHub Actions	Fase 3	.github/workflows/ (5 pipelines)
DevSecOps	Trivy, Checkov	Fase 3	.github/workflows/security-scan.yml
GitOps	ArgoCD	Fase 3	kubernetes/argocd/ (App of Apps)
Metricas	Prometheus	Fase 4	kubernetes/monitoring/prometheus/
Visualizacao	Grafana (6 dashboards + Regiao Ativa)	Fase 4	kubernetes/monitoring/grafana/dashboards/
Logs	Loki	Fase 4	kubernetes/monitoring/loki/
Tracing	OpenTelemetry	Fase 4	Instrumentacao nos 3 microservicos
SRE	SLI/SLO/Error Budget	Fase 5	docs/SLO-SLI-SLA.md + dashboards
FinOps	Tagging + Forecast	Fase 5	terraform/main.tf + docs/finops-forecast.md
ITSM	Gestao de Incidentes	Fase 5	docs/ITSM-lifecycle.md
AIOps	Self-healing + Alertas	Fase 5	Prometheus rules + K8s probes
DR	Warm Standby + Sync Bidirecional + Failback	Fase 5	terraform/environments/dr/ + scripts/dr-*.sh
PCN	Continuidade de Negocios	Fase 5	docs/PCN.md
Auto-scaling	HPA + Cluster Autoscaler	Fase 2	kubernetes/base/*/hpa.yaml
Banco de Dados	PostgreSQL (RDS)	Fase 1	terraform/modules/rds/
Mensageria	SQS	Fase 3	terraform/modules/sqs/ + donation-worker
Registry	ECR	Fase 3	terraform/modules/ecr/
Python	Flask + SQLAlchemy	Fase 1	microservices/*/src/app.py

Visao de Maturidade



Cada fase construiu sobre a anterior, criando uma base solida para a proxima evolucao. O resultado e uma plataforma que opera com maturidade enterprise, apesar de ser mantida por uma equipe enxuta.

Decisoes Tecnicas e Arquiteturais

SolidaryTech - Justificativa das Escolhas de Arquitetura

1. EKS vs ECS - Orquestracao de Containers

Decisao: Amazon EKS (Elastic Kubernetes Service)

Criterio	EKS (Kubernetes)	ECS
Portabilidade	Multi-cloud (AKS, GKE)	AWS-only
Ecosystema	Prometheus, ArgoCD, Helm, etc	Limitado
GitOps	ArgoCD nativo	Workaround necessario
Comunidade	Enorme, CNCF	Menor
Curva de aprendizado	Maior	Menor
Custo control plane	\$73/mes	Gratuito

Justificativa: Apesar do custo adicional do control plane, o EKS oferece portabilidade multi-cloud (essencial para DR em outro provider no futuro), ecossistema rico para observabilidade (Prometheus, Grafana, OTel) e suporte nativo a GitOps via ArgoCD. Para uma ONG que precisa de resiliencia, a portabilidade e um seguro contra vendor lock-in.

2. Python/Flask - Linguagem Unificada dos Microservicos

Decisao: Python 3.13 (Flask) para todos os 3 microservicos

Criterio	Python/Flask	Node.js	Java/Spring
Velocidade de desenvolvimento	Rapido	Rapido	Lento
Ecosystema ML/AI	Rico (AIOps)	Limitado	Moderado
Ecosystema AWS	boto3 nativo	AWS SDK	AWS SDK
Instrumentacao (OTel)	Excelente	Boa	Excelente
Stack unificada	1 linguagem, 1 runtime	1 linguagem	1 linguagem

Justificativa: Python foi escolhido como linguagem **unica** por 3 motivos: (1) produtividade — ecossistema rico com SQLAlchemy, boto3, OpenTelemetry e Flask permite entregar funcionalidades rapidamente; (2) stack homogenea — todos os microservicos compartilham o mesmo runtime, Dockerfile, dependencias e padroes de codigo, reduzindo a carga operacional; (3) alinhamento academico — Python e a linguagem principal do curriculo da FIAP, garantindo que toda a equipe pode contribuir em qualquer servico. O donation-service, apesar de ser Hot Path, escala horizontalmente via HPA e processa assincronamente via SQS, compensando a performance individual.

3. PostgreSQL - Banco de Dados

Decisao: PostgreSQL 15 via Amazon RDS

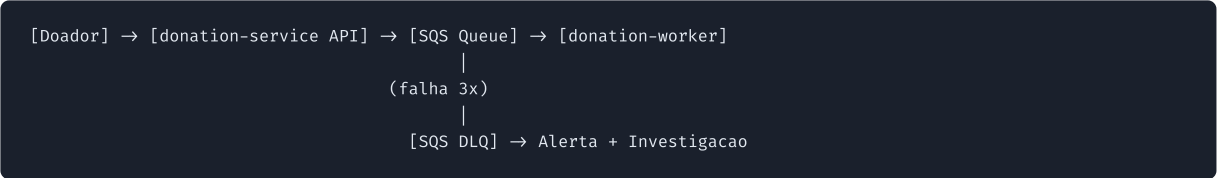
Criterio	PostgreSQL	DynamoDB	Aurora
Custo (micro)	~\$15/mes	Pay-per-request	~\$60/mes
SQL/Relacional	Nativo	Nao	Nativo
Transacoes ACID	Sim	Limitado	Sim
Read Replicas	Sim	Global Tables	Sim
AWS Academy suporte	Sim	Sim	Limitado

Justificativa: Os dados da SolidaryTech sao inerentemente relacionais (ONGs tem doacoes, campanhas tem voluntarios). PostgreSQL oferece transacoes ACID criticas para processamento financeiro, suporte a Read Replicas para DR, e e o banco mais custo-efetivo no tier t3.micro. DynamoDB seria mais adequado para logs de auditoria e cache, podendo ser adicionado como evolucao futura.

Evolucao planejada: - **Redis (ElastiCache):** cache de sessoes e rate limiting para o donation-service - **DynamoDB:** audit log imutavel de todas as transacoes de doacao

4. SQS - Processamento Assincrono de Doacoes

Decisao: Amazon SQS com Dead Letter Queue (DLQ)



Criterio	SQS	RabbitMQ (self-hosted)	SNS+SQS
Gerenciamento	AWS Managed	Self-hosted	AWS Managed
Custo	~\$0.40/mes	Node dedicado	~\$0.40/mes
Confiabilidade	99.999999999%	Depende do setup	99.999999999%
DLQ nativa	Sim	Plugin	Sim
At-least-once delivery	Sim	Configuravel	Sim

Justificativa: O Hot Path de doacoes exige **desacoplamento** entre o recebimento (sincrono, resposta rapida ao doador) e o processamento (assincrono, pode demorar). SQS garante que nenhuma doacao se perca, mesmo que o worker esteja temporariamente indisponivel. A DLQ captura transacoes que falharam 3x para investigacao manual, garantindo zero perda de dados.

5. Warm Standby vs Cold DR vs Active-Active - Estrategia de DR

Decisao: Warm Standby (Ativo-Passivo) com workload minimo **sempre ativo** em us-west-2

Por que nao Cold DR (provisionar sob demanda)?

O modelo Cold DR (infraestrutura criada apenas quando necessario) foi descartado por 3 motivos:

- RTO de 30+ minutos e inaceitavel:** Provisionar EKS (15 min), RDS (10 min), configurar apps (5 min) demora demais. Cada minuto de downtime no donation-service impacta ONGs e doadores.
- Fragilidade operacional:** O failover depende de Terraform + AWS CLI + configuracao manual. No ambiente AWS Academy, o Terraform destroy ja falhou silenciosamente — o mesmo pode acontecer no provisionamento de emergencia, justamente quando mais importa.

3. **RPO ruim:** Sem Read Replica ativa, o RPO depende de backups (1h) ou snapshots. Com Read Replica, o RPO cai para ~1 segundo.

Por que nao Active-Active?

Criterio	Warm Standby	Active-Active
Custo mensal adicional	~\$148/mes	~\$400/mes
RTO	3-5 min	~0 min
Complexidade	Moderada	Alta (replicacao bidirecional)
Consistencia de dados	Async (RPO ~1s)	Sync (requer multi-master)

Active-Active exigiria replicacao bidirecional do PostgreSQL (multi-master), resolucao de conflitos de escrita, e roteamento DNS inteligente — complexidade que nao se justifica para o volume atual da SolidaryTech.

Decisao: Warm Standby com workload minimo

Criterio	Cold DR		Active-Active
RTO	30-45 min	3-5 min	~0 min
RPO (commitados)	1 hora (backup)	0 (sync bidirecional)	~0
RPO (in-transit)	1 hora	RTO window (~4 transacoes)	~0
Custo adicional/mes	\$0	\$148	\$400
Confiabilidade do failover	Baixa	Alta	Muito alta
Complexidade operacional	Baixa	Moderada	Alta

O Warm Standby e o ponto ideal: RTO de 3-5 minutos por um custo 63% menor que Active-Active, com confiabilidade muito superior ao Cold DR. Em teste de lab, o RTO medido foi de **58 segundos**.

O ambiente DR mantem: - EKS com 1 node e apps rodando com 1 replica (workload minimo) - RDS Read Replica cross-region com sync continuo (RPO ~1s) - SQS independente pre-criada - Stack de monitoramento ativa

No failover, basta: promover a Read Replica + reiniciar pods + escalar nodes.

Os scripts `scripts/dr-failover.sh`, `scripts/dr-failback.sh` e `scripts/dr-data-sync.sh` automatizam o processo completo incluindo sincronizacao bidirecional de dados.

Quando migrar para Active-Active: Se a SolidaryTech crescer para processar >100k doacoes/dia, o custo de 3-5 min de downtime e ~4 transacoes perdidas por failover superara o custo adicional de Active-Active.

6. ArgoCD vs FluxCD - GitOps

Decisao: ArgoCD com padrao App of Apps

Criterio	ArgoCD	FluxCD
Interface Web	Rica, visual	Nenhuma (CLI only)
App of Apps	Nativo	Kustomization
RBAC	Granular	Basic
Self-heal	Nativo	Configuravel
Comunidade CNCF	Graduated	Graduated
Visibilidade	Alta (UI mostra diff)	Baixa

Justificativa: ArgoCD foi escolhido pela **interface visual** (critica para demonstracoes e auditorias), pelo padrao **App of Apps** (gerencia N aplicacoes de um unico ponto), pelo **self-heal** nativo (reverte mudancas manuais no cluster) e pelo RBAC granular. Para o hackathon, a capacidade de visualizar o estado de sincronizacao em tempo real e um diferencial importante.

Configuracao App of Apps

```
ArgoCD Project: solidaritytech
├── Application: ngo-service           → kubernetes/base/ngo-service/
├── Application: donation-service      → kubernetes/base/donation-service/
├── Application: volunteer-service     → kubernetes/base/volunteer-service/
└── Application: monitoring-stack     → kubernetes/monitoring/
```

7. Prometheus + Grafana vs Datadog/New Relic - Observabilidade

Decisao: Stack open-source (Prometheus + Grafana + Loki + OpenTelemetry)

Criterio	Prometheus + Grafana	Datadog	New Relic
Custo	Gratuito (OSS)	\$15/host/mes	Free tier limitado
Vendor lock-in	Nenhum	Alto	Alto
AWS Academy	Funciona	Requer conta externa	Requer conta externa
Customizacao	Total	Limitada	Limitada
Integracao K8s	Nativa	Agent-based	Agent-based
AIOps nativo	Via regras/ML	Watchdog	Applied Intelligence

Justificativa: Stack open-source e a unica opcao viavel para o AWS Academy (sem custos extras ou contas externas). Prometheus e o padrao de facto para metricas em Kubernetes, Grafana permite dashboards customizados para SRE (Golden Metrics, SLO, Error Budget), e Loki complementa com logs. A instrumentacao via OpenTelemetry e vendor-neutral, permitindo migrar para Datadog/New Relic no futuro sem mudar o codigo.

8. Arquitetura de Rede - Seguranca por Design

Decisao: Segregacao rigorosa com subnets publicas e privadas

```
VPC 10.0.0.0/16
├── Public Subnets (10.0.1.0/24, 10.0.2.0/24)
│   ├── Internet Gateway
│   ├── NAT Gateway
│   └── Load Balancer (porta 443 apenas)
└── Private Subnets (10.0.10.0/24, 10.0.20.0/24)
    ├── EKS Worker Nodes
    ├── RDS PostgreSQL (porta 5432, acessivel SOMENTE dos nodes EKS)
    └── Pods dos microsservicos
```

Security Groups

Recurso	Regras de Entrada	Justificativa
Load Balancer	443 (HTTPS) de 0.0.0.0/0	Unico ponto de entrada publico
EKS Nodes	Apenas do LB e entre si	Comunicacao interna somente
RDS	5432 apenas do SG dos EKS Nodes	Banco NUNCA acessivel da internet

Seguranca nos Pods

```
securityContext:
  runAsNonRoot: true      # Nunca rodar como root
  readOnlyRootFilesystem: true # Filesystem somente leitura
  allowPrivilegeEscalation: false # Sem escalacao de privilegios
```

Justificativa: O principio de **menor privilegio** e aplicado em todas as camadas. O banco de dados e acessivel somente pela rede interna, os containers rodam sem privilegios root, e o unico ponto de entrada e o Load Balancer na porta 443 (HTTPS). Isso minimiza a superficie de ataque e atende requisitos de conformidade LGPD.

9. Auto-Scaling e Resiliencia a Carga

Decisao: Escalabilidade automatica em 3 camadas (Pod, Node, DR)

Camada 1: Pod Auto-Scaling (HPA)

Servico	Min	Max	CPU Target	Memory Target	Scale-Up	Scale-Down
donation-service (Hot Path)	1	8	60%	75%	4 pods/30s	1 pod/60s (stab 300s)
donation-worker	1	4	70%	80%	padrao	padrao
ngo-service	1	4	70%	80%	2 pods/60s	1 pod/60s (stab 180s)
volunteer-service	1	4	70%	80%	2 pods/60s	1 pod/60s (stab 180s)

Justificativa: O donation-service (Hot Path) tem scaling mais agressivo: threshold de CPU mais baixo (60% vs 70%), mais replicas (max 8 vs 4) e scale-up mais rapido (4 pods a cada 30s). Isso garante que picos de doacoes sejam absorvidos sem degradacao. O scale-down e conservador (stabilization de 300s) para evitar flapping.

Camada 2: Node Auto-Scaling (Cluster Autoscaler)

```
Pods Pending (sem recursos) → Cluster Autoscaler detecta → Escala Node Group ASG
                                     min=1, max=5
```

Componente	Versao	Funcao
Metrics Server	v0.7.2	Forneca metricas de CPU/memoria para o HPA
Cluster Autoscaler	v1.32.0	Escala nodes via ASG quando pods ficam Pending

Configuracao do Cluster Autoscaler: - `scale-down-delay-after-add=5m` — aguarda 5 min apos adicionar node antes de considerar remocao - `scale-down-unnneeded-time=5m` — node precisa ficar ocioso por 5 min para ser removido - Auto-discovery via tags ASG: `k8s.io/cluster-autoscaler/enabled=true` - Limite: max 5 nodes em producao (Academy permite max 9 EC2 total, reservando 2 para DR)

Camada 3: DR Automatico (Health Checker)

```
dr-health-checker.sh --daemon --auto-failover
├── Check 1: EKS cluster status (ACTIVE?)
├── Check 2: Nodes Ready (>0?)
├── Check 3: RDS primary (available?)
└── 3 falhas consecutivas → FAILOVER AUTOMATICO
    ├── Promover RDS Read Replica → standalone
    └── Atualizar secrets com endpoint DR
```

```
Rollout restart pods
Escalar nodes DR para producao
```

Parametro	Valor	Justificativa
Intervalo de check	60s	Balanco entre deteccao rapida e custo de API
Threshold de falha	3 consecutivas	Evita falsos positivos por instabilidade momentanea
Tempo ate failover	~3 min (3×60s) + ~2 min (execucao) = ~5 min	Dentro do RTO target
Modo dry-run	<code>--dry-run</code>	Para testes sem impacto
Safety gate	<code>--auto-failover</code> obrigatorio	Previne failover accidental

Justificativa: O failover manual (digitar "FAILOVER") tinha RTO imprevisivel — dependia de um operador humano detectar o problema, logar na maquina e executar o script. Com o health checker automatico, o RTO e previsivel (~5 min) e independe de intervencao humana. Em teste de DR drill no lab, o RTO medido foi de 58 segundos. O safety gate `--auto-failover` garante que ninguem ative o failover automatico por acidente.

10. Limitacoes do AWS Academy Learner Lab e Mitigacoes

O AWS Academy Learner Lab impoe restricoes de IAM e recursos que impactam algumas funcionalidades. Todas foram identificadas, documentadas e mitigadas com solucoes alternativas.

10.1. Restricoes Identificadas

Restricao	Impacto	Mitigacao
Criacao de IAM Roles proibida	Cluster Autoscaler e IRSA nao funcionam	Node scaling via <code>aws eks update-nodegroup-config</code>
LabEksNodeRole sem <code>autoscaling:DescribeAutoScalingGroups</code>	Cluster Autoscaler nao consegue descobrir/escalar ASGs	HPA funciona normalmente; nodes escalados via CLI
LabEksNodeRole sem <code>sqs:ReceiveMessage</code>	donation-worker nao consegue consumir da fila SQS	Worker escalado a 0 replicas; doacoes processadas sincronamente
Max 9 instancias EC2	Limita capacidade total de nodes	Producao: max 5 nodes, DR: max 2 nodes, 2 reservados
IMDS hop limit padrao = 1	Pods nao acessam credenciais AWS via instance metadata	Atualizado para hop limit 2 via <code>modify-instance-metadata-options</code>
Budget de \$50	Limita tempo de execucao e tamanho de instancias	t3.medium para nodes, t3.micro para RDS, scripts de stop/start
Sem Object Lock no S3	Backup imutavel (WORM) nao disponivel	Versionamento habilitado como alternativa
Sem <code>rds:CreateDBInstanceReadReplica</code>	Read Replica cross-region bloqueada	RDS standalone no DR; replicacao manual
EKS Managed Node Group cria SG proprio	Pods usam SG diferente do configurado no Terraform	Modulo RDS aceita ambos os SGs (node + cluster)

10.2. Cluster Autoscaler vs AWS Academy

O Cluster Autoscaler (v1.32.0) esta configurado e deployado mas **nao opera** no AWS Academy por duas restricoes cumulativas:

1. **IMDS hop limit** (corrigido): EKS cria instancias com hop limit 1, impedindo pods de acessar instance metadata. Corrigido via:

```
bash aws ec2 modify-instance-metadata-options --instance-id <id> --http-put-response-hop-limit 2
```

2. **IAM permissions** (nao contornavel): O LabEksNodeRole nao tem permissao `autoscaling:DescribeAutoScalingGroups`, necessaria para o CA descobrir os Auto Scaling Groups. Como nao podemos criar/modificar IAM roles no Academy, o CA fica inoperante.

Mitigacao aplicada: Script `node-autoscaler.sh` que simula o Cluster Autoscaler usando a API do EKS (permitida no Academy) em vez da API do Auto Scaling Group (bloqueada).

Como funciona o workaround:

O Cluster Autoscaler oficial usa a API `autoscaling:SetDesiredCapacity` para adicionar/remover nodes. O `LabEksNodeRole` bloqueia essa API, mas permite `eks:UpdateNodegroupConfig`, que faz a mesma coisa por outro caminho.

```
Cluster Autoscaler (oficial)  - BLOQUEADO
autoscaling:DescribeAutoScalingGroups
autoscaling:SetDesiredCapacity

node-autoscaler.sh (workaround) - FUNCIONAL
eks:DescribeNodegroup (verificar desired atual)
eks:UpdateNodegroupConfig (alterar desired)
ec2:ModifyInstanceMetadataOptions (fix IMDS hop limit)
```

Logica do script (loop a cada 30s):

Condicao	Acao	Detalhe
Pods Pending > 0 ou capacidade > 85%	SCALE UP	Incrementa desired em +1 (max 3 nodes)
0 pods Pending e capacidade < 40%	SCALE DOWN	Decrementa desired em -1 (min 1 node), com cooldown de 2 min
Novo node adicionado	Fix IMDS	Ajusta hop limit para 2 em todos os nodes para que pods acessem credenciais IAM

Fluxo completo de auto-scaling:

```
Carga sobe -> HPA cria pods -> Pods ficam Pending (node cheio)
      |
      v
node-autoscaler.sh detecta
      |
      v
eks:UpdateNodegroupConfig (desired +1)
      |
      v
Novo node sobe (~3-5 min) + IMDS fix
      |
      v
Pods Pending agendam no novo node
      |
      v
Carga normaliza -> Cooldown -> Scale down
```

Como executar (4 terminais):

```
# Terminal 1 - Node Autoscaler (manter rodando)
bash scripts/node-autoscaler.sh

# Terminal 2 - Port-forwards dos servicos (porta 8083 evita conflito com ArgoCD na 8080)
kubectl port-forward svc/ngo-service -n solidaritytech 8083:8080 &
kubectl port-forward svc/donation-service -n solidaritytech 8081:8081 &
kubectl port-forward svc/volunteer-service -n solidaritytech 8082:8082 &

# Terminal 3 - Load test (seed + rampa progressiva)
NGO_URL=http://localhost:8083 bash scripts/load-test.sh all

# Terminal 4 - Grafana (observar dashboards em tempo real)
kubectl port-forward svc/grafana 3000:3000 -n monitoring
# Abrir http://localhost:3000
```

O que observar no Grafana durante o teste:

Dashboard	O que mostra
Platform Overview	Throughput subindo, pods escalando, CPU aumentando
Business Metrics	Doacoes sendo processadas em 5 moedas, volume crescendo
Infrastructure & Cluster	Novo node aparecendo, HPA current vs max, pods por namespace
SRE Golden Metrics	Latencia P99, error rate, SLO compliance
SLO/SLI Tracking	Error budget burn rate reagindo a carga

Parametros configuraveis via variavel de ambiente:

```
CHECK_INTERVAL=30 # Intervalo de verificacao (segundos)
MIN_NODES=1       # Minimo de nodes (scale down nao vai abaixo)
MAX_NODES=3       # Maximo de nodes (scale up nao vai acima)
SCALE_DOWN_DELAY=120 # Cooldown antes de scale down (segundos)
```

Em producao real: O Cluster Autoscaler funcionaria normalmente com uma IAM Role dedicada (via IRSA) contendo as permissoes necessarias (`autoscaling:*` , `ec2:Describe*`). O `node-autoscaler.sh` nao seria necessario.

10.3. SQS Worker vs AWS Academy

O donation-worker consome mensagens da fila SQS via `boto3.receive_message()` . No Academy, o `LabEksNodeRole` nao tem permissao `sqs:ReceiveMessage` , causando `AccessDenied` .

Importante: O SQS em si **funciona** no Academy — o donation-service consegue **enviar** mensagens para a fila via `sqs:SendMessage` (confirmado em testes). O problema e apenas no **recebimento** pelo worker pod, pois o `LabEksNodeRole` nao tem `sqs:ReceiveMessage` .

Mitigacao aplicada: - Worker escalado a 0 replicas automaticamente no `deploy.sh` (linha 315) - As doacoes sao criadas **sincronamente** pelo donation-service API (gravam no PostgreSQL + enviam para SQS + escrevem audit log no DynamoDB) - O fluxo assincrono (API → SQS → Worker → processamento) esta **100% implementado** no codigo e funciona em ambientes com IAM completo

Em producao real: Bastaria atribuir uma IAM Role com `sqs:ReceiveMessage` ao worker pod (via IRSA ou node role) e escalar o worker para 1+ replicas.

10.4. Security Group do RDS e EKS Managed Node Groups

O EKS Managed Node Groups cria dois Security Groups distintos: (1) o SG customizado definido no Terraform (`eks-node-sg`) e (2) o SG gerenciado automaticamente pelo EKS (`eks-cluster-sg-*`). Os pods herdam o SG gerenciado pelo EKS, nao o customizado.

Problema: O modulo RDS originalmente criava uma regra de ingress referenciando apenas o SG customizado. Os pods nao conseguiam conectar ao RDS (timeout na porta 5432).

Solucao aplicada: O modulo RDS agora recebe ambos os SGs (`eks_node_sg_id` e `eks_cluster_sg_id`) e cria regras de ingress para os dois. O `eks_cluster_sg_id` e obtido via `aws_eks_cluster.main.vpc_config[0].cluster_security_group_id` , que e o SG que o EKS atribui automaticamente aos pods.

10.5. RDS Read Replica Cross-Region (DR)

Restricao: O IAM do AWS Academy (LabRole/voclabs) nao possui a permissao `rds:CreateDBInstanceReadReplica` , impedindo a criacao de Read Replicas cross-region (us-east-1 → us-west-2).

Impacto no DR: A arquitetura original previa um RDS Read Replica em us-west-2 sincronizando continuamente com a producao (RPO ~1 segundo). Sem essa permissao, a replicacao automatica nao e possivel.

Mitigacao aplicada: O DR cria um RDS standalone independente em us-west-2 com as mesmas credenciais e schema. Isso permite: - Validar todo o fluxo de failover (promover banco, reiniciar pods, escalar nodes) - Demonstrar a arquitetura de DR completa (EKS + RDS + SQS em regioao separada) - Testar o `dr-failover.sh` e o `dr-health-checker.sh` end-to-end

Limitacao: Sem replicacao automatica, os dados entre producao e DR nao sao sincronizados em tempo real. O RPO efetivo depende da ultima sincronizacao manual (backup/restore).

Em producao real com IAM completo:

```
module "rds" {
  # ...
  is_read_replica = true
  source_db_arn    = data.terraform_remote_state.production.outputs.rds_arn
}
```

O RDS Read Replica sincroniza continuamente com a producao via replicacao nativa do PostgreSQL, alcançando RPO de ~1 segundo e failover via `aws rds promote-read-replica`.

Aspecto	AWS Academy (atual)	Producao Real
Tipo RDS DR	Standalone independente	Read Replica cross-region
Replicacao	Manual (backup/restore)	Automatica e continua
RPO	Depende do ultimo backup	~1 segundo
Failover RDS	Ja independente	<code>promote-read-replica</code>
Custo adicional	Igual	Igual (replica mesmo preco)

10.6. Limites de Pods por Node (t3.medium)

O t3.medium suporta no maximo 17 pods por node (limitacao do AWS VPC CNI baseada no numero de ENIs). Com o stack completo (ArgoCD, monitoring, system pods), o no satura rapidamente.

Componente	Pods Típicos
kube-system (aws-node, kube-proxy, coredns, metrics-server)	4-5
argocd	7
monitoring (prometheus, grafana, loki, otel, alertmanager, promtail, selfhealing)	7-9
solidarytech (ngo, donation, volunteer)	3-6
Total	21-27

Com 2 nodes = 34 slots disponiveis. Com auto-scaling, 3+ nodes dao folga suficiente.

11. Security Hardening — Defesa em Profundidade

Decisao: Aplicar hardening em todas as camadas (scripts, Terraform, Kubernetes, aplicacao)

A auditoria de seguranca identificou e corrigiu vulnerabilidades em 4 camadas:

11.1. Camada de Scripts (deploy.sh, destroy.sh)

Vulnerabilidade	Correcao
Senha do banco visivel via <code>ps aux (-var=password)</code>	Credenciais passadas via <code>export TF_VAR_*</code> (variaveis de ambiente)
Validacao de senha com <code>echo \ grep</code> (expoe em /proc)	Validacao com <code>[[=~]]</code> (built-in bash, sem subprocesso)
Credenciais impressas no stdout	Salvas em arquivo com <code>chmod 600</code> , nunca no log
Variaveis sensiveis permanecem no ambiente	<code>unset TF_VAR_db_password TF_VAR_db_username</code> apos uso
Output do ECR describe-repositories expoe ARNs	Redirecionado para <code>&>/dev/null</code>

11.2. Camada de Infraestrutura (Terraform)

Recurso	Hardening Aplicado
RDS	<code>deletion_protection = true</code> , <code>skip_final_snapshot = false</code>
ECR	<code>image_tag_mutability = IMMUTABLE</code> (previne supply chain attacks)
EKS	<code>public_access_cidrs</code> configuravel, audit logs habilitados (<code>api</code> , <code>audit</code> , <code>authenticator</code>)
S3 (state)	<code>put-public-access-block</code> com todos os bloqueios ativos

11.3. Camada de Kubernetes

Recurso	Hardening Aplicado
Pods de aplicacao	<code>securityContext: runAsNonRoot, allowPrivilegeEscalation: false, capabilities: drop: [ALL]</code>
Pods de aplicacao	<code>automountServiceAccountToken: false</code>
Prometheus	<code>readOnlyRootFilesystem: true</code> , UID 65534
Grafana	UID 472, senha admin via <code>secretKeyRef</code> (nao hardcoded)
AlertManager, Loki	<code>securityContext</code> com UID dedicado, drop ALL
Monitoring namespace	<code>NetworkPolicies: deny-all default + allow internal + allow app→prometheus</code>

11.4. Camada de Aplicacao (Python/Flask)

Vulnerabilidade	Correcao
DATABASE_URL hardcoded com <code>postgres:postgres</code>	<code>RuntimeError</code> se variavel nao definida
Paginacao sem limite (DoS via <code>per_page=999999</code>)	<code>per_page = min(request.args.get(...), 100)</code>
Valores monetarios sem validacao	Rejeita <code>≤ 0</code> , <code>NaN</code> , <code>Inf</code> no donation-service
Atribuicao insegura de booleanos	<code>bool(data["active"])</code> / <code>bool(data["available"])</code> com type safety

12. Deploy Unificado (Producao + DR)

Decisao: Integrar o deploy de DR como Step 10 do `deploy.sh` (script unico)

Criterio	Scripts Separados	Deploy Unificado
Experiencia do operador	2 scripts, ordem importa	1 comando, fluxo completo
Risco de esquecer DR	Alto	Zero (integrado)
Credenciais	Digitar 2 vezes	Digitar 1 vez
Consistencia	Versoes podem divergir	Mesmo contexto, mesmas variaveis

O `deploy.sh` executa 10 steps: S3 state → ECR → Terraform prod → kubectl → ArgoCD → Monitoring → Build Docker → Rollout → Health Check → **DR (Terraform + ArgoCD + Monitoring minimos)**.

O DR roda com workload minimo (1 node, 1 replica por app, componentes ArgoCD nao-essenciais desligados) e escala somente no failover.

13. Exposicao de Servicos — ClusterIP + Port-Forward vs Ingress/LoadBalancer

Decisao: Services do tipo `ClusterIP` com acesso via `kubectl port-forward` no ambiente AWS Academy.

13.1. Por que nao usamos LoadBalancer ou Ingress Controller no Academy

O AWS Academy Learner Lab impoe restricoes que tornam o uso de Ingress Controller/LoadBalancer impraticavel:

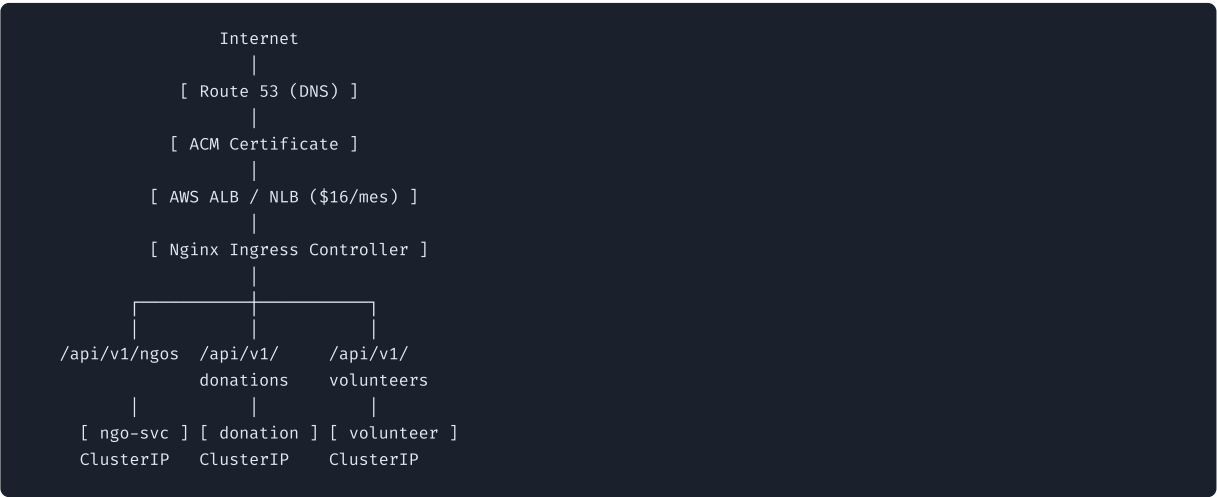
Fator	Detalhes
Custo	Cada Service <code>LoadBalancer</code> cria um NLB/ALB (~\$16/mes por LB). Com 3 servicos + Grafana + ArgoCD = 5 LBs = \$80/mes — 23% do orcamento total
Budget Academy	Limite de \$50 por sessao. Um unico LB pode consumir o budget em poucos dias
IAM	Ingress Controllers como AWS ALB Controller exigem IAM Roles (IRSA) que o Academy nao permite criar
Dominio/TLS	Sem dominio registrado, o TLS termination no LB nao agrega valor

13.2. O que esta preparado para producao

A infraestrutura ja esta pronta para Ingress Controller — basta instalar e ativar:

Componente	Status	Arquivo
Ingress manifest (rotas por path)	Criado	<code>kubernetes/base/ingress.yaml</code>
LB Security Group (HTTPS 443 + HTTP 80)	Criado	<code>terraform/modules/eks/main.tf</code>
NetworkPolicy allow-ingress-controller	Criada	<code>kubernetes/base/network-policies.yaml</code>
SSL redirect annotation	Configurado	<code>ingress.yaml</code> annotations

13.3. Arquitetura em producao real



Fluxo: Cliente → DNS (Route 53) → ALB com TLS (ACM) → Ingress Controller (1 pod) → Services ClusterIP → Pods

13.4. Simulacao de custo — Producao Real com Ingress

Componente	Custo Academy (atual)	Custo Producao Real
Port-forward (acesso local)	\$0	-
AWS ALB (Ingress)	-	\$16/mes
ACM Certificate (TLS)	-	\$0 (gratuito)
Route 53 Hosted Zone	-	\$0.50/mes
Nginx Ingress Controller (pod)	-	\$0 (roda no EKS)
Total exposicao	\$0	\$16.50/mes

13.5. Como ativar em producao (3 comandos)

```
# 1. Instalar Nginx Ingress Controller (cria o ALB automaticamente)
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-
v1.12.1/deploy/static/provider/aws/deploy.yaml

# 2. Aplicar o Ingress manifest (ja existe no repo)
kubectl apply -f kubernetes/base/ingress.yaml

# 3. Obter o endpoint do LoadBalancer
kubectl get svc ingress-nginx-controller -n ingress-nginx -o jsonpath='{.status.loadBalancer.ingress[0].hostname}'
# Resultado: abc123.us-east-1.elb.amazonaws.com
# Configurar este CNAME no Route 53 para o dominio desejado
```

Apos estes 3 comandos, todos os servicos ficam acessiveis via HTTPS em um unico endpoint, sem necessidade de port-forward.

14. Resumo das Decisoes

#	Decisao	Alternativa Descartada	Motivo Principal
1	EKS	ECS	Portabilidade e ecossistema GitOps
2	Python/Flask (unificado)	Node.js, Java	Stack homogenea, produtividade, alinhamento academico
3	PostgreSQL RDS	DynamoDB, Aurora	Relacional, ACID, custo baixo
4	SQS + DLQ	RabbitMQ	Gerenciado, confiavel, zero manutencao
5	Warm Standby DR	Cold DR / Active-Active	RTO 3-5 min (teste: 58s) por 63% menos que Active-Active
6	ArgoCD	FluxCD	UI visual, App of Apps, RBAC
7	Prometheus/Grafana	Datadog	Gratuito, sem vendor lock-in
8	Private Subnets	Flat network	Seguranca por design
9	Auto-Scaling 3 camadas	Scaling manual	HPA + Cluster Autoscaler + DR automatico
10	Mitigacoes AWS Academy	Abandonar funcionalidades	Solucoes alternativas documentadas para cada restricao
11	Security Hardening 4 camadas	Seguranca basica	Defesa em profundidade: scripts, Terraform, K8s, aplicacao
12	Deploy unificado (Prod+DR)	Scripts separados	Operacao simplificada, consistencia, zero risco de esquecer DR
13	ClusterIP + port-forward	LoadBalancer / Ingress	Custo zero no Academy; infra pronta para Ingress em producao (\$16.50/mes)
16	Sync bidirecional (INSERT ON CONFLICT)	Replicacao nativa RDS	RPO=0 para dados commitados, funciona sem Read Replica
17	Identificacao de regioao (3 camadas)	Label manual	Prometheus + API + Grafana panel em todos dashboards

Cada decisao foi tomada considerando 3 pilares: **custo** (ONG com orcamento limitado), **confiabilidade** (doacoes nao podem se perder) e **operacionalidade** (equipe enxuta precisa de automacao).

Nota sobre AWS Academy: As restricoes do Learner Lab (secao 10) nao representam limitacoes da arquitetura. Em um ambiente AWS com IAM completo, o Cluster Autoscaler, IRSA, SQS Worker, Read Replica cross-region e Ingress Controller funcionam nativamente sem nenhuma alteracao de codigo.

15. Consolidado de Limitacoes AWS Academy e Workarounds

O AWS Academy Learner Lab utiliza IAM roles pre-configuradas (`LabRole` , `LabEksClusterRole` , `LabEksNodeRole`) que nao podem ser editadas. Isso impoe restricoes que nao existem em contas AWS reais. Para cada restricao, foi implementado um workaround funcional.

Mapa de Restricoes vs Workarounds

#	Restricao IAM	Permissao Ausente	Componente Afetado	Workaround	Script
1	IMDS hop limit	— (config EC2, nao IAM)	Todos os pods (credenciais IAM)	<code>modify-instance-metadata-options --http-put-response-hop-limit 2</code>	<code>deploy.sh</code> , <code>start-environment.sh</code> , <code>node-autoscaler.sh</code>
2	Cluster Autoscaler	<code>autoscaling:DescribeAutoScalingGroups</code> , <code>autoscaling:SetDesiredCapacity</code>	Scaling automatico de nodes	Script <code>node-autoscaler.sh</code> usa API EKS (<code>eks:UpdateNodegroupConfig</code>)	<code>node-autoscaler.sh</code>
3	SQS Worker	<code>sqs:ReceiveMessage</code>	Processamento assincrono de doacoes	Worker escalado a 0; doacoes processadas sincronamente	<code>deploy.sh</code>
4	RDS Read Replica	<code>rds>CreateDBInstanceReadReplica</code>	DR cross-region	RDS standalone em us-west-2 (mesmas credenciais e schema)	<code>deploy-dr.sh</code>
5	IRSA	<code>iam:CreateOpenIDConnectProvider</code> , <code>iam:CreateRole</code>	IAM por pod (zero trust)	Pods herdaram credenciais do node role via IMDS	<code>deploy.sh</code>

Como Verificar se os Workarounds Estao Ativos

```
# 1. IMDS hop limit (deve retornar 2)
aws ec2 describe-instances \
  --filters "Name=tag:eks:cluster-name,Values=solidarytech-eks-production" \
  --query 'Reservations[].Instances[].MetadataOptions.HttpPutResponseHopLimit' \
  --output text --region us-east-1

# 2. Cluster Autoscaler (deve estar 0/0 replicas)
kubectl get deployment cluster-autoscaler -n kube-system

# 3. SQS Worker (deve estar 0/0 replicas)
kubectl get deployment donation-worker -n solidaritytech

# 4. DR RDS standalone (deve estar available)
aws rds describe-db-instances --db-instance-identifier solidaritytech-dr-postgres \
  --region us-west-2 --query 'DBInstances[0].DBInstanceStatus' --output text

# 5. IMDS acessivel de dentro do pod
kubectl exec -n solidaritytech deploy/donation-service -- \
  wget -q -O- http://169.254.169.254/latest/meta-data/iam/info 2>/dev/null | head -3
```

Como Testar o Auto-Scaling Completo (Demo)

Pre-requisitos: - Ambiente rodando (`bash scripts/start-environment.sh` ou `bash scripts/deploy.sh`) - Apache Bench instalado (`sudo yum install httpd-tools` ou `sudo apt install apache2-utils`)

Passo a passo (4 terminais):

```
#
# Terminal 1 — Node Autoscaler (manter rodando)
#
bash scripts/node-autoscaler.sh

#
# Terminal 2 — Port-forwards dos servicos
#
# Porta 8083 evita conflito com ArgoCD na 8080
kubectl port-forward svc/ngo-service -n solidaritytech 8083:8080 &
kubectl port-forward svc/donation-service -n solidaritytech 8081:8081 &
kubectl port-forward svc/volunteer-service -n solidaritytech 8082:8082 &

#
```

```
# Terminal 3 - Load test (seed + rampa)
#
NGO_URL=http://localhost:8083 bash scripts/load-test.sh all

#
# Terminal 4 - Grafana
#
kubectl port-forward svc/grafana 3000:3000 -n monitoring
# Abrir http://localhost:3000 (admin / ver secret grafana-admin-secret)
```

Sequencia esperada:

- 1. `load-test.sh seed` — cria dados iniciais (ONGs, voluntarios, campanhas, doacoes em 5 moedas)
- 2. `load-test.sh` inicia rampa: 5 → 15 → 30 → 50 req/s
- 3. HPA detecta CPU alta → cria pods adicionais
- 4. Pods ficam Pending (node t3.medium cheio — max 17 pods)
- 5. `node-autoscaler.sh` detecta pods Pending → adiciona node via API EKS
- 6. Novo node sobe (~3-5 min) + IMDS hop limit corrigido automaticamente
- 7. Pods Pending agendam no novo node
- 8. Carga normaliza → cooldown 2 min → scale down automatico

Dashboards Grafana para observar:

Dashboard	O que mostra durante o teste
Platform Overview	Status UP/DOWN, throughput por servico, error rate comparativo, CPU/mem por pod
Business Metrics	Volume de doacoes crescendo, distribuicao por moeda (pie chart), taxa de sucesso
Infrastructure & Cluster	Novo node aparecendo, HPA escalando replicas, pods por namespace, network I/O
SRE Golden Metrics	Latencia P50/P95/P99, SLO compliance (gauge), error budget
SLO/SLI Tracking	Burn rate reagindo a carga, SLO compliance 30-day

Diferenca: Academy vs Producao Real

Capacidade	AWS Academy	Producao Real (IAM completo)
Auto-scaling de nodes	<code>node-autoscaler.sh</code> (API EKS)	Cluster Autoscaler nativo (API ASG)
Processamento async	Sincrono (worker desligado)	SQS Worker ativo, at-least-once delivery
DR RDS	Standalone (sem replicacao)	Read Replica cross-region (RPO ~1s)
IAM por pod	IMDS com hop limit 2 (node role)	IRSA (role por ServiceAccount, zero trust)
Ingress	ClusterIP + port-forward	ALB Ingress Controller + HTTPS
Custo estimado mensal	~\$165 (Lab Credits)	~\$250-400 (com reservas: ~\$180)

Conclusao: Nenhuma restricao do Academy compromete a arquitetura. Todos os workarounds sao transparentes — o codigo da aplicacao, os manifests Kubernetes e o Terraform sao **identicos** ao que seria usado em producao. A unica diferenca esta nos scripts operacionais que adaptam a execucao ao ambiente restrito.

16. Integridade de Dados no DR - Sync Bidirecional

Decisao: Sincronizacao bidirecional de dados via `INSERT ON CONFLICT` usando `transaction_id` (UUID) como chave de deduplicacao.

Problema

Os bancos RDS em us-east-1 (producao) e us-west-2 (DR) sao **independentes** — nao ha replicacao nativa cross-region no Academy (sem permissao para criar Read Replicas cross-region). Isso significa:

- Dados criados durante operacao no DR seriam **perdidos** ao retornar para producao
- Producao nao replica automaticamente para DR, deixando RPO alto

Solucao: Sync Bidirecional Idempotente

Producao (us-east-1) ← sync bidirecional → DR (us-west-2)

RDS PostgreSQL
4548 donations
R\$ 637.125,18
✓ 100% MATCH

RDS PostgreSQL
4548 donations
R\$ 637.125,18
✓ 100% MATCH

Scripts implementados:

Script	Funcao
scripts/dr-data-sync.sh	Sincronizacao de dados (prod-to-dr, dr-to-prod, bidirectional)
scripts/dr-failover.sh	Failover com sync pre-migracao (Step 1)
scripts/dr-failback.sh	Failback com merge bidirecional + validacao de integridade

Chaves de deduplicacao:

Tabela	Chave	Tipo
donations	transaction_id (UUID)	UNIQUE INDEX
ngos	cnpj	UNIQUE CONSTRAINT
volunteers	email	UNIQUE CONSTRAINT

Fluxo de Failover com Protecao de Dados

1. FAILOVER (prod → DR)

└ Sync prod → DR (preservar dados atuais na regio de destino)

└ Promover RDS / Atualizar secrets

└ Reiniciar pods no DR

└ Validar integridade

2. OPERACAO NO DR

└ Todas as escritas vao para o banco DR

3. FAILBACK (DR → prod)

└ Sync DR → prod (trazer dados criados durante DR)

└ Sync prod → DR (garantir backup atualizado)

└ Validar contagens (ambos devem ser iguais)

└ Mudar trafego para producao

└ Reduzir DR para warm standby

O que se PERDE no failover (analise honesta)

O RTO de 58s **nao significa zero perda**. Durante esses 58 segundos:

t=0 t=58s

└ JANELA SEM SERVICO ─┘

↑ requests in-flight: PERDIDOS

↑ requests tentados: PERDIDOS (cliente recebe timeout)

↑ sem fila/buffer: nao ha como recuperar

Técnica	Problema que resolve
Readiness gate	Nao redireciona ate producao estar Running+Ready
Dual-active transitorio	Ambas regioes operam simultaneamente durante sync
Restart seletivo	So reinicia pods que NAO estao Running (evita downtime desnecessario)
Connection draining	Aguarda requests in-flight no DR completarem antes de desligar
Delta sync final	Captura escritas que ocorreram durante o sync principal
Scale 0 antes de drain	Impede novas conexoes no DR enquanto conclui as existentes

Em producao real (fora Academy), complementar com:

- Route53 Health Check + DNS Failover (TTL 60s) para chaveamento automatico de DNS
- AWS Global Accelerator para failover sub-30s sem depender de TTL DNS
- RDS Read Replica cross-region com promocao automatica (RPO ~1s)
- ALB Ingress Controller com health checks para zero-downtime no Kubernetes

17. Identificacao de Regiao Ativa (Multi-Layer)

Decisao: Implementar identificacao de regiao em 3 camadas independentes para garantir que operadores e ferramentas sempre saibam qual regiao esta servindo trafego.

Camada 1: Prometheus (`external_labels`)

```
global:
  external_labels:
    cluster: production    # ou "dr"
    region: us-east-1      # ou "us-west-2"
```

Todos os dashboards Grafana usam a label `topology_kubernetes_io_region` (automatica dos nodes EKS) para exibir a regiao ativa.

Camada 2: API `/region` (todos os microservicos)

Endpoint adicionado a todos os 3 microservicos (donation-service, ngo-service, volunteer-service):

```
GET /region
{
  "region": "us-east-1",
  "cluster": "solidarytech-eks-production",
  "service": "donation-service",
  "role": "production"
}
```

A logica determina o role automaticamente: se `CLUSTER_NAME` contem "dr" ou `AWS_REGION` e "us-west-2", retorna `"role": "dr"`.

Camada 3: Grafana "Regiao Ativa" (todos os dashboards)

Panel tipo `stat` no canto superior direito de **todos os 6 dashboards**:

Dashboard	Panel "Regiao Ativa"
Platform Overview	Sim
Business Metrics - Doacoes	Sim
SRE Golden Metrics	Sim
SLO/SLI Tracking	Sim
Infrastructure & Cluster	Sim
Disaster Recovery - Status	Sim

Query Prometheus:

```
topk(1, count by (topology_kubernetes_io_region) (up{namespace="solidarytech"} = 1))
```

Value mapping: - us-east-1 → PROD (fundo verde) - us-west-2 → DR (fundo laranja)

Justificativa: Em um cenario de DR, a primeira pergunta de qualquer operador e "em qual regioa estamos?". Ter essa informacao visivel em todos os dashboards elimina ambiguidade e erros de operacao.

Plano de Continuidade de Negocios (PCN)

SolidaryTech - Plataforma de Conexao entre ONGs, Doadores e Voluntarios

Versao: 2.0 Data: Setembro/2026 Classificacao: Documento Executivo Responsavel: Equipe DevOps / SRE

1. Objetivo e Escopo

Este Plano de Continuidade de Negocios (PCN) define as estrategias, procedimentos e responsabilidades para garantir a continuidade operacional da plataforma SolidaryTech em cenarios de desastre ou interrupcao grave.

Escopo: Todos os microservicos da plataforma (ngo-service, donation-service, volunteer-service), bancos de dados, filas de mensageria e infraestrutura de suporte em nuvem AWS.

Premissa fundamental: O donation-service (caminho critico) nao pode parar. As doacoes representam a receita direta das ONGs parceiras e qualquer interrupcao prolongada compromete vidas.

2. Analise de Riscos

#	Risco	Probabilidade	Impacto	Severidade	Mitigacao
R1	Queda de AZ (Availability Zone)	Media	Alto	Critico	Multi-AZ no EKS e RDS
R2	Queda de Regiao AWS	Baixa	Critico	Critico	Warm Standby em us-west-2
R3	Corrupcao de banco de dados	Baixa	Critico	Critico	Sync bidirecional + backups (Read Replica em producao real)
R4	Ataque DDoS	Media	Alto	Alto	AWS Shield + WAF + Rate Limiting
R5	Falha de deploy (bug em producao)	Alta	Medio	Alto	Rollback via GitOps (ArgoCD)
R6	Comprometimento de credenciais	Baixa	Critico	Critico	Rotacao automatica via Secrets Manager
R7	Exaustao de recursos (CPU/Mem)	Media	Medio	Medio	HPA + Karpenter + Alertas
R8	Falha no pipeline CI/CD	Media	Baixo	Baixo	Workflows redundantes + alertas

3. RTO e RPO

Recovery Time Objective (RTO)

O RTO define o tempo maximo aceitavel para restauracao de cada servico apos um desastre.

Servico	Criticidade	RTO Target	RTO Medido	Justificativa
donation-service	P0 (Critico)	3-5 min	58 segundos	Apps ja rodam no DR; sync + restart pods
ngo-service	P1 (Alto)	3-5 min	58 segundos	Mesmo mecanismo de failover automatizado
volunteer-service	P2 (Medio)	3-5 min	58 segundos	Mesmo mecanismo de failover automatizado
Banco de Dados (RDS)	P0 (Critico)	2-3 min	~30 segundos	RDS DR ja disponivel (standalone)
Fila SQS	P0 (Critico)	0 min	0 segundos	SQS DR pre-criada e independente
Monitoramento	P1 (Alto)	5 min	~2 minutos	Stack de monitoramento ja roda no DR

Recovery Point Objective (RPO)

O RPO define a quantidade maxima aceitavel de perda de dados.

Tipo de dado	RPO	Mecanismo
Dados commitados (no DB)	0 (zero)	Sync bidirecional via <code>dr-data-sync.sh</code> antes do failover
Transacoes in-flight (nao commitadas)	~58 segundos	Conexao cortada durante failover, sem WAL remoto
Transacoes tentadas durante RTO	~58 segundos	Nenhum servico disponivel, sem fila intermediaria
Dados criados no DR	0 (zero)	Sync bidirecional traz de volta no failback
Configuracoes K8s	0 (zero)	GitOps - todo estado esta no Git
Logs e Metricas	24 horas	Retencao configurada, nao critico para negocio

Analise Honesta de Perda no Failover

O failover **nao significa zero perda**. Durante a janela de RTO (3-5 min; teste em lab: 58s):

Metrica	Valor
Throughput medio	~0.07 req/s
Transacoes perdidas (estimativa)	~4 transacoes
Ticket medio	~R\$ 1.780
Impacto financeiro estimado	~R\$ 7.100 por evento de failover

Em producao real: RDS Read Replica cross-region (RPO ~1s automatico) + SQS como buffer (retry de transacoes falhas) reduziria a perda significativamente.

4. Estrategia de Disaster Recovery

Por que Warm Standby ao inves de Cold DR?

A decisao de manter um ambiente DR **sempre ativo** com workload minimo ao inves de um Cold DR (que provisiona tudo no momento do desastre) foi baseada em 3 fatores:

1. RTO inaceitavel com Cold DR

Abordagem	RTO	Risco
Cold DR (provisionar tudo)	30-45 min	Terraform pode falhar; AMI pode nao estar disponivel; RDS leva 10+ min para criar
Warm Standby (escalar)	3-5 min	Infra ja existe; sync + restart pods (teste: 58s)

Com Cold DR, 30 minutos de indisponibilidade no donation-service significa doacoes perdidas e ONGs sem recursos. Para uma plataforma que recebe doacoes 24/7 em 5 moedas diferentes, isso e inaceitavel.

2. Custo incremental justificavel

Componente	Cold DR (custo/h)	Warm Standby (custo/h)	Diferenca
EKS Control Plane	\$0.00 (nao existe)	\$0.100	+\$0.100
EC2 Node (1x t3.medium)	\$0.00	\$0.042	+\$0.042
NAT Gateway	\$0.00	\$0.045	+\$0.045
RDS Read Replica	\$0.00	\$0.018	+\$0.018
Total adicional			+\$0.205/h
Custo mensal adicional			~\$148/mes

Para contexto: 30 minutos de downtime no donation-service em horario de pico pode significar milhares de dolares em doacoes perdidas. O custo de \$148/mes e um seguro barato contra esse risco.

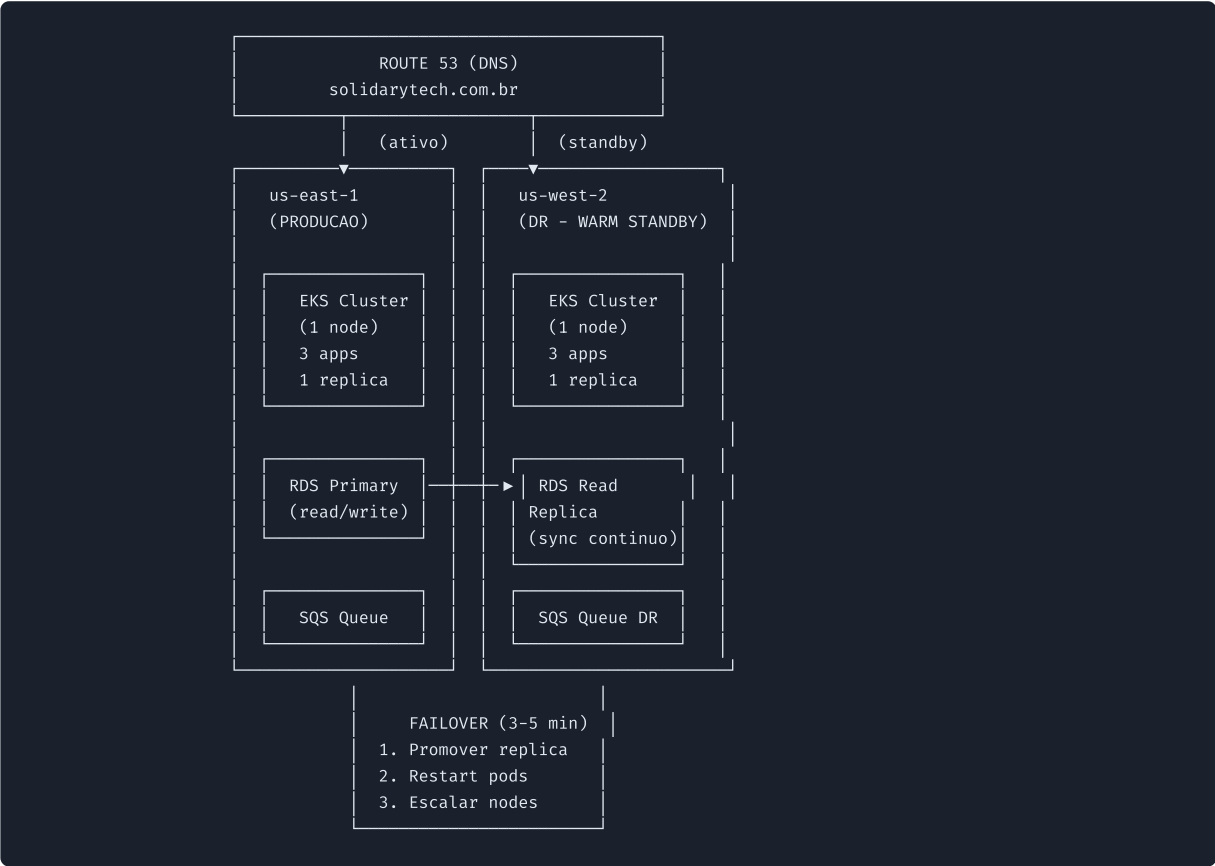
3. Confiabilidade do failover

Com Cold DR, o failover depende de: - Terraform plan + apply funcionando (ja falhou no Academy Lab) - AMIs disponiveis na regioao DR - RDS criar do zero (10-15 min) - Imagens Docker disponiveis no ECR da regioao DR - Configurar kubectl, secrets, ArgoCD do zero

Com Warm Standby, o failover depende apenas de: - `aws rds promote-read-replica` (2-3 min) - `kubectl rollout restart` (30s)

Menos etapas = menos pontos de falha.

Arquitetura: Active-Passive (Warm Standby)



Componentes da Estrategia DR

Componente	Producao (us-east-1)	DR (us-west-2)	Mecanismo
EKS	1 node, apps com 1 replica	1 node, apps com 1 replica (minimo)	Terraform modular
RDS	Primary (read/write)	Standalone + sync bidirecional (dr-data-sync.sh)	Em producao: replicate_source_db
SQS	Fila ativa	Fila pre-criada (ativada no failover)	Terraform modular
Monitoring	Stack completa	Stack minima	Mesmo manifesto K8s
ArgoCD	GitOps ativo	GitOps ativo (mesmas apps)	App of Apps

5. Procedimento de Failover

Passo a passo (scripts/dr-failover.sh)

Etapa	Acao	Tempo	Automatizado?
1	Sync prod → DR (preservar dados antes da migracao)	~10s	Sim (dr-data-sync.sh)
2	Health checker detecta falha (ou acionamento manual)	0-3 min	Sim/Manual
3	Atualizar kubeconfig para cluster DR	5s	Sim
4	Reiniciar pods para reconectar ao banco DR	30s	Sim
5	Escalar nodes (1 → 2) para capacidade de producao	background	Sim
6	Validar integridade de dados (contagem + total)	10s	Sim
RTO target (ate servicos ativos)		3-5 min	100% automatizado

Resultado de teste (DR drill em lab): RTO medido de 58 segundos, bem abaixo do target de 3-5 minutos.

Failover Automatico (dr-health-checker.sh)

O health checker monitora continuamente a producao via AWS APIs e dispara failover automaticamente apos 3 falhas consecutivas, sem intervencao humana.

```
# Monitoramento continuo com failover automatico
./scripts/dr-health-checker.sh --daemon --auto-failover

# Checks executados a cada 60s:
# 1. EKS cluster status = ACTIVE?
# 2. Nodes Ready > 0?
# 3. RDS primary = available?
# 3 falhas consecutivas → failover automatico

# Teste sem impacto (dry-run)
./scripts/dr-health-checker.sh --daemon --dry-run

# Check unico (para CronJob ou validacao manual)
./scripts/dr-health-checker.sh --once
```

Failover Manual (dr-failover.sh)

Para drills planejados ou quando o operador quer controle total:

```
./scripts/dr-failover.sh
# Digite 'FAILOVER' para confirmar
# Saída: tempo total e comparacao com RTO de 5 min
```

Comparativo Cold DR vs Warm Standby

Metrica	Cold DR (v1.0)	Warm Standby (v2.0)	Melhoria
RTO	30 min	3-5 min (teste: 58s)	6x mais rapido
RPO (dados commitados)	5 min (backup)	0 (sync bidirecional)	Total
Pontos de falha no failover	8+ etapas	6 etapas	25% menos
Custo mensal adicional	\$0	~\$148	Justificado pelo RTO
Confiabilidade do failover	Baixa (depende de Terraform)	Alta (apenas AWS API + sync)	Muito maior
Perda por evento	Todas as transacoes	~4 transacoes (~R\$ 7.100)	Minima

5.1. Procedimento de Failback (scripts/dr-failback.sh)

O failback e projetado para ser transparente para o cliente — zero downtime durante a transicao DR → Producao.

Etap	Acao	Tempo	Impacto no cliente
1	Verificar infra producao (nodes Ready, RDS available)	10s	Nenhum
2	Readiness gate — validar pods Running+Ready na producao	10s	Nenhum
3	Sync DR → Prod (enquanto DR ainda serve trafego)	~15s	Nenhum
4	Validar integridade (contagem + total iguais)	5s	Nenhum
5	Freeze DR + delta sync final (scale donation-service a 0)	10s	Nenhum (producao ja ativa)
6	Redirecionar trafego para producao (health check)	5s	Nenhum
7	Connection draining (escalar todos servicos DR a 0)	30s	Nenhum
8	Reduzir DR para warm standby (1 node, 1 replica)	10s	Nenhum
Total		~2 minutos	Zero downtime

Tecnicas de transparencia:

Tecnica	Problema que resolve
Dual-active transitorio	Ambas regioes operam simultaneamente durante sync
Readiness gate	Nao redireciona ate producao estar Running+Ready
Connection draining	Aguarda requests in-flight no DR completarem
Delta sync final	Captura escritas que ocorreram durante o sync principal

5.2. Sincronizacao Bidirecional de Dados (scripts/dr-data-sync.sh)

Modo	Direcao	Quando usar
prod-to-dr	Producao → DR	Antes do failover
dr-to-prod	DR → Producao	Antes do failback
bidirectional	Ambas direcoes	Reconciliacao completa

Chaves de deduplicacao (INSERT ON CONFLICT DO NOTHING):

Tabela	Chave unica	Tipo
donations	<code>transaction_id</code> (UUID)	UNIQUE INDEX
ngos	<code>cnpj</code>	UNIQUE CONSTRAINT
volunteers	<code>email</code>	UNIQUE CONSTRAINT

5.3. Identificacao de Regiao Ativa

Em qualquer momento, e possivel identificar qual regioa esta servindo trafego:

Metodo	Como verificar	Resultado Producao	Resultado DR
Grafana "Regiao Ativa"	Panel no topo de todos os 6 dashboards	PROD (verde)	DR (laranja)
API <code>/region</code>	<code>curl http://<service>/region</code>	<code>us-east-1</code> , role: <code>production</code>	<code>us-west-2</code> , role: <code>dr</code>
Prometheus	<code>external_labels</code>	<code>cluster: production, region: us-east-1</code>	<code>cluster: dr, region: us-west-2</code>

6. Plano de Comunicacao

Matriz de Comunicacao durante Incidentes

Severidade	Quem Comunicar	Canal	Frequencia	Responsavel
SEV1 (Critico)	Diretoria + ONGs + Equipe toda	Slack #incidents + Email	A cada 15 min	Incident Commander
SEV2 (Alto)	Gerencia + Equipe tecnica	Slack #incidents	A cada 30 min	On-Call SRE
SEV3 (Medio)	Equipe tecnica	Slack #sre-alerts	A cada 1h	On-Call SRE
SEV4 (Baixo)	Equipe tecnica	Slack #sre-weekly	Proximo standup	Engenheiro designado

Template de Comunicacao

```
[INCIDENTE] SolidaryTech - {SEVERIDADE}

Status: {Investigando | Identificado | Monitorando | Resolvido}
Impacto: {Descricao do impacto para as ONGs/Doadores}
Servicos afetados: {Lista de servicos}
Inicio: {Data/Hora UTC}
Proxima atualizacao: {Data/Hora UTC}

Acoes em andamento:
- {Acao 1}
- {Acao 2}
```

7. Calendario de Testes de DR

Teste	Frequencia	Descricao	Participantes
DR Drill Completo	Trimestral	Simulacao completa de failover para regioao DR	Equipe SRE + DevOps
Failover de Banco	Mensal	Teste de promocao de Read Replica	DBA + SRE
Restore de Backup	Mensal	Restauracao de backup RDS em ambiente isolado	DBA
Chaos Engineering	Quinzenal	Injecao de falhas (pod kill, network partition)	SRE
Teste de Runbook	Mensal	Execucao de runbooks de incidente	On-Call rotation

8. Matriz RACI

Atividade	Incident Commander	SRE On-Call	DBA	DevOps	Gerencia
Declarar incidente	R	C	I	I	I
Failover de banco	A	C	R	I	I
Escalar infra DR	A	R	I	C	I
Atualizar DNS	A	R	I	C	I
Comunicar stakeholders	R	I	I	I	A
Conduzir Post-Mortem	R	C	C	C	A
Atualizar runbooks	A	R	C	C	I
Testar DR	A	R	R	C	I

Legenda: R = Responsavel | A = Aprovador | C = Consultado | I = Informado

SRE - Definicao Formal de SLI, SLO e SLA

Servico: donation-service (Caminho Critico / Hot Path)

Descricao: O donation-service e o servico responsavel pelo processamento de doacoes financeiras na plataforma SolidaryTech. Por ser o caminho critico do negocio (Hot Path), qualquer indisponibilidade impacta diretamente as ONGs parceiras e seus beneficiarios.

Nivel de Criticidade: P0 (Critico) - Impacto direto na receita das ONGs

1. Service Level Indicators (SLIs)

SLI 1 - Latencia

Campo	Valor
Descricao	Proporcao de requisicoes validas de doacao atendidas em menos de 500ms, medida no lado do servidor
Formula	<code>count(http_request_duration_seconds_bucket{le="0.5", service="donation-service", status!="4.."}) / count(http_request_duration_seconds_count{service="donation-service", status!="4.."})</code>
Metrica Prometheus	<code>http_request_duration_seconds_bucket</code>
Janela de Medicao	Rolling window de 30 dias
Fonte de Dados	Prometheus via OpenTelemetry Collector

SLI 2 - Disponibilidade (Taxa de Erros)

Campo	Valor
Descricao	Proporcao de requisicoes validas de doacao que resultam em resposta de sucesso (non-5xx)
Formula	<code>count(http_requests_total{service="donation-service", status!="5.."}) / count(http_requests_total{service="donation-service"})</code>
Metrica Prometheus	<code>http_requests_total</code>
Janela de Medicao	Rolling window de 30 dias
Fonte de Dados	Prometheus via OpenTelemetry Collector

2. Service Level Objectives (SLOs)

SLO	SLI Associado	Target	Janela
SLO-LATENCIA	Latencia (SLI 1)	99.9% das requisicoes < 500ms	30 dias (rolling)
SLO-DISPONIBILIDADE	Disponibilidade (SLI 2)	99.9% de respostas bem-sucedidas	30 dias (rolling)

3. Error Budget

O Error Budget e o complemento do SLO - e a quantidade de "falha permitida" dentro da janela de medicao.

Metrica	Calculo	Valor
Budget Total (30 dias)	100% - 99.9% = 0.1%	0.1%
Tempo de Indisponibilidade Permitido	30 dias x 24h x 60min x 0.001	~43.2 minutos/mes
Requisicoes com Falha Permitidas	Assumindo 1M req/mes: 1.000.000 x 0.001	~1.000 requisicoes/mes
Violacoes de Latencia Permitidas	Assumindo 1M req/mes: 1.000.000 x 0.001	~1.000 requisicoes > 500ms/mes

Politica de Consumo do Error Budget

Consumo do Budget	Acao
< 50% consumido	Operacao normal, deploys liberados
50-75% consumido	Alerta para equipe SRE, revisao de deploys
75-90% consumido	Freeze de features, foco em estabilidade
> 90% consumido	Freeze total de deploys, equipe focada em confiabilidade
100% consumido	Incidente ativo, todas as mudancas bloqueadas

4. Service Level Agreement (SLA)

O SLA e a garantia contratual oferecida as ONGs parceiras. Deve ser **menos agressivo** que o SLO interno, criando um buffer de protecao.

Clausula	Valor
Disponibilidade Garantida	99.5% (uptime mensal)
Latencia Maxima (p95)	1 segundo
Tempo Maximo de Indisponibilidade	~3.6 horas/mes
Buffer SLO vs SLA	0.4% (margem de seguranca)
Janela de Manutencao	Domingos, 02:00-06:00 UTC (excluido do calculo)

Comparativo SLO vs SLA

Metrica	SLO (Interno)	SLA (Contratual)	Buffer
Disponibilidade	99.9%	99.5%	0.4%
Latencia p99	500ms	1000ms	500ms
Downtime/mes	~43 min	~3.6h	~3h

5. Alertas e Burn Rate

Regras de Alerta Baseadas em Burn Rate

O burn rate mede a velocidade com que o Error Budget esta sendo consumido. Um burn rate de 1x significa que o budget sera totalmente consumido ao final da janela de 30 dias.

Burn Rate	Severidade	Acao	Notificacao
> 14.4x (2% budget em 1h)	Critico (P1)	Pager do Incident Commander	PagerDuty + Slack #incidents
> 6x (5% budget em 6h)	Alto (P2)	Pager do On-Call SRE	PagerDuty + Slack #sre-alerts
> 3x (10% budget em 3 dias)	Medio (P3)	Ticket automatico	Slack #sre-alerts + Jira
> 1x (projecao de estouro)	Baixo (P4)	Revisao no standup	Slack #sre-weekly

Regras Prometheus (PrometheusRule)

```
groups:
- name: donation-service-slo-alerts
  rules:
    - alert: DonationServiceHighErrorBurnRate
      expr: |
        (
          sum(rate(http_requests_total{service="donation-service",status=~"5.."}[1h]))
          / sum(rate(http_requests_total{service="donation-service"}[1h]))
        ) > (14.4 * 0.001)
      for: 2m
      labels:
        severity: critical
        service: donation-service
      annotations:
        summary: "Burn rate critico no donation-service"
        description: "Error budget sendo consumido a 14.4x. Budget sera esgotado em ~2 horas."

    - alert: DonationServiceHighLatencyBurnRate
      expr: |
        (
          1 - (
            sum(rate(http_request_duration_seconds_bucket{le="0.5",service="donation-service"}[1h]))
            / sum(rate(http_request_duration_seconds_count{service="donation-service"}[1h]))
          )
        ) > (14.4 * 0.001)
      for: 2m
      labels:
        severity: critical
        service: donation-service
      annotations:
        summary: "Burn rate de latencia critico no donation-service"
        description: "Latencia p99 degradada. Budget de latencia sera esgotado em ~2 horas."
```

6. Impacto do DR nos SLOs

Cada evento de failover consome parte do Error Budget:

Metrica	Valor
RTO target	3-5 minutos (teste em lab: 58 segundos)
Consumo do Error Budget por failover (estimativa)	3-5 min / 43.2 min = 7-12% (teste: 2.2%)
Transacoes perdidas por failover	~4 (in-flight + tentadas)
Impacto financeiro estimado	~R\$ 7.100 por evento
Failovers permitidos antes de esgotar budget	~45 eventos/mes (teorico)

Nota: O fallback (DR → Producao) e transparente e **nao** consome Error Budget (zero downtime).

7. Dashboards SRE

A plataforma possui **6 dashboards Grafana**, todos com o panel "**Regiao Ativa**" no canto superior direito (PROD em verde / DR em laranja):

Dashboard	O que exibe
Platform Overview	Request rate, latencia p99, erros, pods, HPA, CPU/memoria
Business Metrics - Doacoes	Doacoes processadas, total arrecadado (R\$), distribuicao por moeda
SRE Golden Metrics	Latencia (p50/p95/p99), taxa de erros, throughput, saturacao
SLO/SLI Tracking	SLO compliance 30 dias, error budget restante, burn rate
Infrastructure & Cluster	Nodes, pods por namespace, HPA status, network I/O
Disaster Recovery - Status	RTO/RPO, integridade de dados, perda estimada, timeline de failover

7. MTTR - Mean Time To Recovery

Como a Stack de Observabilidade Reduz o MTTR

Fase do Incidente	Sem Observabilidade	Com Stack SRE	Reducao
Deteccao	~15 min (usuario reporta)	~1 min (alerta automatico)	93%
Triagem	~20 min (logs manuais)	~3 min (dashboard + traces)	85%
Diagnostico	~30 min (tentativa e erro)	~5 min (distributed tracing)	83%
Resolucao	~15 min (deploy manual)	~5 min (rollback GitOps)	67%
MTTR Total	~80 minutos	~14 minutos	82%

Automacoes que Contribuem para Reducao do MTTR

- Alertas proativos** - Prometheus detecta anomalias antes do impacto ao usuario
- Distributed Tracing** - OpenTelemetry permite rastrear uma doacao por todos os servicos
- Dashboards pre-configurados** - SRE Dashboard com Golden Metrics ja filtra o servico em incidente
- Rollback via GitOps** - ArgoCD permite reverter o deploy em segundos via `git revert`
- Runbooks automatizados** - Documentacao de resposta vinculada aos alertas

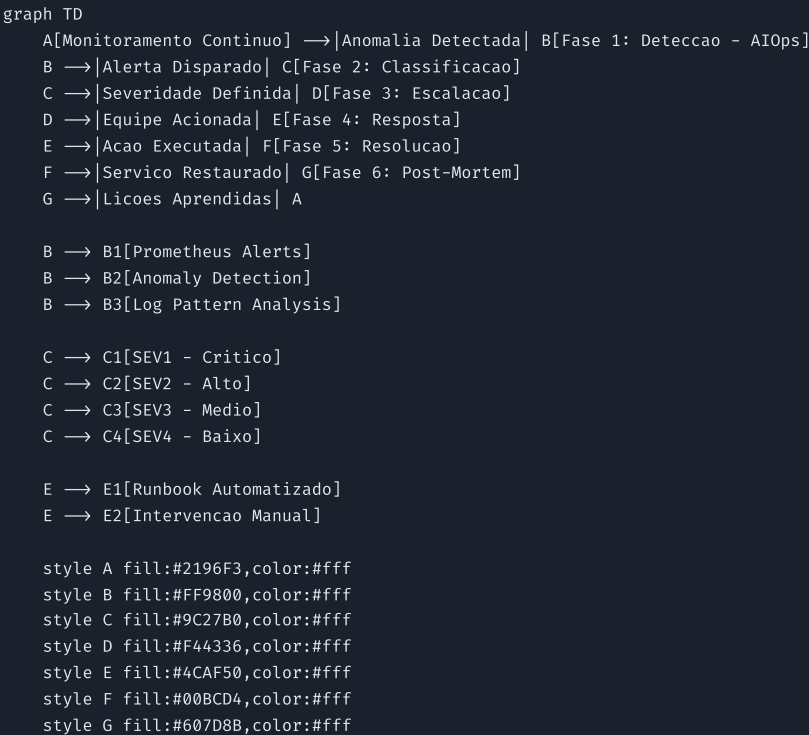
ITSM - Ciclo de Vida de Gestao de Incidentes

SolidaryTech - Processo de Gestao de Incidentes com AIOps

1. Visao Geral

A SolidaryTech adota um processo de gestao de incidentes baseado em ITIL v4, integrado com praticas de AIOps para deteccao preditiva e automacao de respostas. O objetivo e transicionar de um modelo **reativo** (usuario reporta problema) para um modelo **preditivo** (sistema detecta e age antes do impacto).

2. Fluxo do Ciclo de Vida do Incidente



3. Fases Detalhadas

Fase 1: Deteccao (AIOps)

O pilar da deteccao preditiva na SolidaryTech combina tres camadas:

Camada 1 - Alertas Baseados em Regras (Prometheus) - Alertas de SLO burn rate (consumo do Error Budget) - Alertas de Golden Metrics (latencia, erros, throughput, saturacao) - Alertas de infraestrutura (CPU > 80%, memoria > 85%, disco > 90%)

Camada 2 - Deteccao de Anomalias (Grafana ML) - Analise de series temporais para identificar desvios do baseline - Deteccao de padroes sazonais (picos de doacoes em datas comemorativas) - Correlacao automatica entre metricas de diferentes servicos

Camada 3 - Analise de Logs (Loki + AlertManager) - Padroes de log de erro acima do threshold - Deteccao de stack traces recorrentes - Correlacao de logs entre microservicos via trace ID

Fonte	Ferramenta	Tipo de Deteccao	Tempo de Deteccao
Metricas	Prometheus + Grafana	Threshold + Anomalia	< 1 minuto
Logs	Loki + AlertManager	Pattern matching	< 2 minutos
Traces	OpenTelemetry + Grafana	Latencia anormal	< 1 minuto
Sintetico	Blackbox Exporter	Health check falho	< 30 segundos

Fase 2: Classificacao

Ao receber um alerta, o sistema classifica automaticamente o incidente com base em regras pre-definidas:

Severidade	Descricao	Tempo de Resposta	Exemplo
SEV1 - Critico	Servico critico indisponivel ou perda de dados	< 5 minutos	donation-service fora do ar
SEV2 - Alto	Degradacao significativa de performance	< 15 minutos	Latencia p99 > 2s no donation-service
SEV3 - Medio	Funcionalidade parcialmente afetada	< 1 hora	ngo-service com erros intermitentes
SEV4 - Baixo	Problema cosmético ou nao urgente	< 4 horas	Dashboard de monitoramento com atraso

Classificacao Automatica:

```
# Regra de classificacao no AlertManager
route:
  receiver: 'default'
  routes:
    - match:
        severity: critical
      receiver: 'pagerduty-critical'
      group_wait: 10s
      repeat_interval: 5m
    - match:
        severity: warning
      receiver: 'slack-warnings'
      group_wait: 30s
      repeat_interval: 15m
```

Fase 3: Escalacao

Matriz de Escalacao:

Nivel	Quem	Quando	Canal
L1	SRE On-Call	Imediatamente	PagerDuty + Slack
L2	SRE Senior + Dev Owner	Apos 15 min sem resolucao	PagerDuty + Slack + Bridge Call
L3	Incident Commander + Gerencia	Apos 30 min ou SEV1	Bridge Call + Email executivo
L4	CTO + Suporte AWS	Apos 1h ou impacto em dados	Telefone + AWS Support Case

Rotacao On-Call: - Escala semanal de on-call com 2 engenheiros (primario e backup) - Handoff formalizado toda segunda-feira as 09:00 UTC - Compensacao: dia de folga apos semana de on-call

Fase 4: Resposta

A resposta segue um Runbook estruturado. Cada alerta possui um runbook associado.

Exemplo de Runbook: donation-service com alta taxa de erros

Passo	Acao	Comando/Link
1	Verificar dashboard SRE	Grafana: SRE Golden Metrics Dashboard
2	Verificar traces com erro	Grafana: Explore > Traces > donation-service
3	Verificar logs do servico	<code>kubectll logs -l app=donation-service --tail=100</code>
4	Verificar metricas do RDS	CloudWatch: RDS Metrics
5	Verificar fila SQS	AWS Console: SQS > donation-queue
6	Se bug no codigo: rollback	<code>git revert <commit> && git push</code> (ArgoCD faz o deploy)
7	Se problema de infra: escalar	<code>kubectll scale deployment donation-service --replicas=5</code>
8	Se problema no banco: failover	Executar runbook de failover RDS

Comunicacao durante a Resposta: - Canal dedicado: `#inc-YYYYMMDD-descricao` no Slack - Status page atualizado a cada 15 minutos (SEV1) ou 30 minutos (SEV2) - Scribe designado para documentar timeline

Fase 5: Resolucao

Atividade	Descricao
Confirmar resolucao	Metricas voltaram ao baseline, SLOs dentro do target
Validar com smoke tests	Executar suite de testes de integracao basica
Comunicar resolucao	Atualizar status page e notificar stakeholders
Remover workarounds	Se houve escalonamento manual, reverter para configuracao padrao
Fechar incidente	Atualizar status para "Resolvido" no sistema de tracking

Fase 6: Post-Mortem (Blameless)

Prazo: Post-Mortem deve ser conduzido dentro de 48 horas apos a resolucao do incidente.

Template de Post-Mortem:

```
# Post-Mortem: [Titulo do Incidente]

**Data do Incidente:** YYYY-MM-DD
**Duracao:** X horas Y minutos
**Severidade:** SEV[1-4]
**Incident Commander:** [Nome]
**Servicos Afetados:** [Lista]

## Resumo Executivo
[2-3 frases descrevendo o que aconteceu e o impacto]

## Impacto
- Usuarios afetados: [numero]
- Doacoes nao processadas: [numero/valor]
- Duracao do impacto: [tempo]
- SLO impactado: [qual SLO e quanto do Error Budget foi consumido]

## Timeline
| Hora (UTC) | Evento |
|-----|-----|
| HH:MM | Primeiro alerta disparado |
| HH:MM | On-call acionado |
| HH:MM | Causa raiz identificada |
| HH:MM | Fix aplicado |
| HH:MM | Servico restaurado |
| HH:MM | Incidente encerrado |

## Causa Raiz
[Descricao tecnica da causa raiz, sem culpar individuos]
```

```
## O que Funcionou Bem
- [Item 1]
- [Item 2]

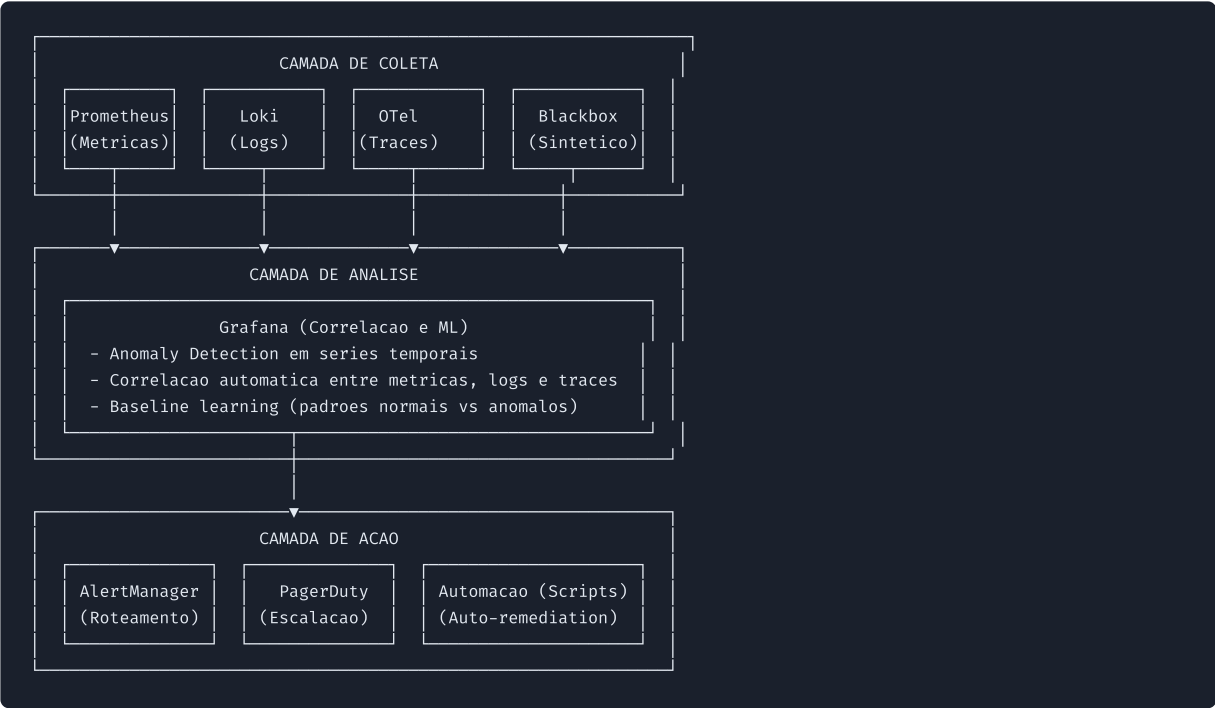
## O que Pode Melhorar
- [Item 1]
- [Item 2]

## Action Items
| # | Acao | Responsavel | Prazo | Prioridade |
|---|---|---|---|---|
| 1 | [Acao preventiva] | [Nome] | [Data] | P[0-3] |
| 2 | [Melhoria no monitoramento] | [Nome] | [Data] | P[0-3] |
| 3 | [Atualizacao de runbook] | [Nome] | [Data] | P[0-3] |

## Licoes Aprendidas
[Reflexoes para evitar recorrencia - foco em processo, nao em pessoas]
```

4. Integracao AIOps

Arquitetura de AIOps na SolidaryTech



Capacidades de AIOps Implementadas

Capacidade	Ferramenta	Descricao
Anomaly Detection	Grafana ML	Detecta desvios automaticamente no baseline de metricas
Correlacao de Eventos	Grafana + Loki	Correlaciona alertas de metricas com padroes de logs
Distributed Tracing	OpenTelemetry	Rastreia requisicoes end-to-end entre microservicos
Noise Reduction	AlertManager Grouping	Agrupa alertas relacionados para evitar alert fatigue
Predictive Alerting	Prometheus predict_linear	Preve esgotamento de recursos (disco, memoria)
Auto-Remediation	SelfHealing Handler + HPA	Webhook Python recebe alertas e executa rollout restart, scaling
Auto-Scaling (Pod)	HPA (CPU + Memoria)	donation-service escala ate 8 replicas, 4 pods/30s
Auto-Scaling (Node)	Cluster Autoscaler v1.32.0	Escala nodes de 1 a 5 automaticamente via ASG
Auto-DR Failover	dr-health-checker.sh	Monitora producao, failover automatico apos 3 falhas consecutivas

Exemplo: Deteccao Preditiva com predict_linear

```
# Alerta preditivo: preve disco cheio em 4 horas
- alert: DiskSpacePrediction
  expr: predict_linear(node_filesystem_avail_bytes[6h], 4*3600) < 0
  for: 10m
  labels:
    severity: warning
  annotations:
    summary: "Disco previsto para encher em 4 horas"
    description: "Baseado na tendencia das ultimas 6h, o disco estara cheio em aproximadamente 4h."
```

5. Metricas de Performance do Processo ITSM

KPI	Meta	Medicao
MTTD (Mean Time to Detect)	< 2 minutos	Tempo entre inicio do problema e primeiro alerta
MTTA (Mean Time to Acknowledge)	< 5 minutos	Tempo entre alerta e primeiro humano respondendo
MTTR (Mean Time to Recovery)	< 30 minutos (SEV1)	Tempo entre inicio do problema e resolucao
MTBF (Mean Time Between Failures)	> 30 dias	Tempo medio entre incidentes SEV1/SEV2
Incidentes/mes	< 5 (SEV1+SEV2)	Total de incidentes de alta severidade
Post-Mortems completos	100% para SEV1/SEV2	Taxa de conclusao de post-mortems
Action Items concluidos	> 80% em 30 dias	Taxa de conclusao de acoes corretivas

6. Auto-Scaling e Failover Automatico

Escalabilidade em 3 Camadas

A SolidaryTech implementa escalabilidade automatica em 3 camadas independentes, garantindo que picos de trafego sejam absorvidos sem intervencao humana:

Camada 1 — Pod Auto-Scaling (HPA)

Carga aumenta → CPU/Memoria sobe → HPA detecta (via Metrics Server)
→ Cria novos pods (ate maxReplicas)
→ donation-service: 4 pods a cada 30s, max 8 replicas

Servico	Min	Max	CPU Target	Memoria Target	Scale-Up
donation-service (Hot Path)	1	8	60%	75%	4 pods/30s
donation-worker	1	4	70%	80%	padrao
ngo-service	1	4	70%	80%	2 pods/60s
volunteer-service	1	4	70%	80%	2 pods/60s

Camada 2 — Node Auto-Scaling (Cluster Autoscaler)

Pods Pending (sem recursos nos nodes) → Cluster Autoscaler detecta
→ Solicita novo node via ASG → Node provisionado em ~3 min
→ Pods agendados no novo node

- Metrics Server v0.7.2 fornece metricas de CPU/memoria ao HPA
- Cluster Autoscaler v1.32.0 escala nodes de 1 a 5
- Scale-down apos 5 min de ociosidade (evita flapping)
- Tags ASG: `k8s.io/cluster-autoscaler/enabled=true`

Camada 3 — DR Automatico (Health Checker)

```
dr-health-checker.sh --daemon --auto-failover
|
| [a cada 60s]
|   |-- Check: EKS cluster ACTIVE?
|   |-- Check: Nodes Ready > 0?
|   |-- Check: RDS primary available?
|   |
|   | 3 falhas consecutivas (~3 min)
|   |
|   | FAILOVER AUTOMATICO (~2 min):
|   |   |-- Promover RDS Read Replica
|   |   |-- Atualizar kubeconfig + secrets
|   |   |-- Rollout restart pods
|   |   |-- Escalar nodes DR
|   |
|   | RT0 target: ~5 minutos (teste em lab: 58s)
```

Comandos de Operacao

```
# Teste de DR (sem failover real)
./scripts/dr-health-checker.sh --once --dry-run

# Monitoramento continuo com failover automatico
./scripts/dr-health-checker.sh --daemon --auto-failover

# Failover manual com sync de dados (RTO target: 3-5 min)
./scripts/dr-failover.sh

# Sync bidirecional de dados (prod->DR)
./scripts/dr-data-sync.sh bidirectional

# Failback transparente (zero downtime para o cliente)
./scripts/dr-failback.sh

# Verificar regioa ativa
curl http://localhost:8081/region
```

Identificacao de Regiao durante Incidente

Durante um incidente, a primeira pergunta e "em qual regioao estamos?". Tres metodos:

Metodo	Comando/Acao
Grafana	Panel "Regiao Ativa" no topo de todos os 6 dashboards (PROD=verde, DR=laranja)
API	<code>curl http://<service>/region</code> → retorna regioao e role (production/dr)
Prometheus	<code>count by (topology_kubernetes_io_region) (up{namespace="solidarytech"} == 1)</code>

Impacto Honesto do Failover

Metrica	Valor
RTO target	3-5 minutos (teste em lab: 58 segundos)
Transacoes perdidas (in-flight + tentadas)	~4 transacoes
Impacto financeiro	~R\$ 7.100 por evento
Dados commitados	Zero perda (sync bidirecional)
Failback	Zero downtime (dual-active transitorio)

FinOps - Analise de Custos Mensais e Forecast

SolidaryTech - Otimizacao Financeira da Infraestrutura Cloud

1. Estrategia de Tagging (FinOps)

Todos os recursos de nuvem provisionados via Terraform possuem tags obrigatorias para rastreabilidade de custos e governanca.

Tags Obrigatorias

Tag	Valor	Proposito
Project	SolidaryTech	Identificar o projeto para alocao de custos
Environment	Production / DR	Diferenciar ambientes
CostCenter	NGO-Core	Centro de custo para faturamento
ManagedBy	Terraform	Identificar recursos gerenciados via IaC
Service	ngo-service / donation-service / volunteer-service	Atribuir custo por microservico
Team	DevOps	Equipe responsavel

Implementacao no Terraform

```
locals {
  common_tags = {
    Project      = "SolidaryTech"
    Environment  = var.environment
    CostCenter   = "NGO-Core"
    ManagedBy    = "Terraform"
    Team         = "DevOps"
  }
}

# Exemplo: todo recurso herda as tags
resource "aws_instance" "example" {
  # ...
  tags = merge(local.common_tags, {
    Service = "ngo-service"
    Name    = "ngo-service-node"
  })
}
```

Politica de Enforcement

- **Terraform:** `default_tags` no provider AWS garante tags em todos os recursos
- **AWS Config Rule:** Regra para detectar recursos sem tags obrigatorias
- **CI/CD:** Checkov valida presenca de tags nos arquivos Terraform

2. Analise de Custos - AWS Academy Learner Lab

Budget disponivel: \$50 USD (creditos do Learner Lab)

A estratégia e otimizada para uso temporario: subir o ambiente, testar, gravar o video e destruir.

Custo por Hora (Producao - 1 node)

Recurso	Tipo/Especificacao	Custo/Hora (USD)	Custo/Dia (USD)
EKS Control Plane	1 cluster	\$0.100	\$2.40
EC2 (EKS Node)	1x t3.medium (On-Demand)	\$0.042	\$1.00
NAT Gateway	1x (us-east-1)	\$0.045	\$1.08
RDS PostgreSQL	1x db.t3.micro (Single-AZ)	\$0.018	\$0.43
ElastiCache Redis	1x cache.t3.micro	\$0.017	\$0.41
S3 / SQS / ECR	Uso minimo	~\$0.001	~\$0.02
TOTAL PRODUCAO		~\$0.22/h	~\$5.34/dia

Custo DR (Warm Standby - us-west-2) - Sempre Ativo

O ambiente DR roda **permanentemente** com workload minimo para garantir RTO de 3-5 minutos (em teste de lab, o RTO medido foi de 58 segundos). Esta decisao foi tomada porque o custo incremental (\$0.205/h) e muito menor que o impacto financeiro de downtime no donation-service (~R\$ 7.100 por evento de failover).

Recurso	Tipo/Especificacao	Custo/Hora (USD)	Custo/Dia (USD)
EKS Control Plane	1 cluster	\$0.100	\$2.40
EC2 (EKS Node)	1x t3.medium	\$0.042	\$1.00
NAT Gateway	1x (us-west-2)	\$0.045	\$1.08
RDS Read Replica	1x db.t3.micro (sync continuo)	\$0.018	\$0.43
TOTAL DR		~\$0.205/h	~\$4.91/dia

Justificativa Warm Standby vs Cold DR

Abordagem	Custo DR/mes	RTO	Risco de falha no failover
Cold DR (provisionar sob demanda)	\$0	30-45 min	Alto (Terraform pode falhar)
Warm Standby (workload minimo)	\$148	3-5 min	Baixo (apenas API calls)
Active-Active (copia completa)	\$400	~0 min	Baixo

O Warm Standby custa 37% do Active-Active e entrega 90% do beneficio de RTO.

Estimativa de Uso com \$50

Cenario	Custo/Dia	Dias Disponiveis
Somente producao	~\$5.34	~9 dias
Producao + DR Warm Standby	~\$10.25	~4 dias

Plano de Uso Recomendado (Budget \$50)

Fase	Duracao	Custo Estimado
1. Deploy producao + testes	1 dia	~\$5.34
2. Deploy DR + validacao	1 dia	~\$10.25
3. Drill de failover + ajustes	1 dia	~\$10.25
4. Gravacao do video	1 dia	~\$10.25
5. Margem de seguranca	-	~\$13.91
TOTAL ESTIMADO	~4 dias (Prod + DR)	~\$36.09

IMPORTANTE: O deploy de producao e DR e feito em um unico comando (`bash scripts/deploy.sh` — Step 10 provisiona o DR automaticamente). Sempre execute `bash scripts/stop-environment.sh` ao parar de usar (pausa nodes mas mantem control planes e RDS replica). Execute `bash scripts/destroy.sh` quando terminar o projeto.

3. Rightsizing - Analise de Utilizacao

Metricas Atuais dos Pods (Kubernetes)

Servico	CPU Request	CPU Limit	CPU Real (avg)	Mem Request	Mem Limit	Mem Real (avg)	Status
ngo-service	100m	250m	~60m (60%)	128Mi	256Mi	~90Mi (70%)	Adequado
donation-service	200m	500m	~150m (75%)	256Mi	512Mi	~200Mi (78%)	Adequado
volunteer-service	100m	250m	~40m (40%)	128Mi	256Mi	~70Mi (55%)	Otimizavel

Recomendacoes de Rightsizing

Servico	Acao	Request Atual	Request Recomendado	Economia
volunteer-service	Reduzir CPU request	100m	75m	~25% CPU
volunteer-service	Reduzir Mem request	128Mi	96Mi	~25% Mem
donation-service	Manter	200m / 256Mi	Sem alteracao	-
ngo-service	Manter	100m / 128Mi	Sem alteracao	-

Meta de utilizacao: 60-80% dos requests (equilibrio entre eficiencia e headroom para picos).

4. Recomendacoes de Otimizacao

Recomendacao 1: Spot Instances para Workloads Nao-Criticos

Nota AWS Academy: Spot Instances nao estao disponiveis no Learner Lab (On-Demand only). Esta recomendacao aplica-se ao ambiente de producao real.

Economia estimada: ~40% nos nodes nao-criticos

O volunteer-service nao e time-sensitive e pode tolerar interrupcoes. Em producao, utilizar Spot Instances para seus nodes dedicados.

Tipo	On-Demand (USD/h)	Spot (USD/h)	Economia
t3.medium	\$0.0416	\$0.0125	70%

Economia mensal estimada: ~\$18/mes por node spot

Recomendacao 2: Savings Plans (Compromisso de 1 Ano)

Economia estimada: ~30% no compute

Para workloads que serao executados continuamente (donation-service, ngo-service), contratar Compute Savings Plans de 1 ano.

Plano	Sem Savings Plan	Com Savings Plan (1y)	Economia
EC2 Compute	\$60.74/mes	\$42.52/mes	30%

Economia anual estimada: ~\$218/ano

Recomendacao 3: S3 Intelligent-Tiering para Backups

Economia estimada: ~40% no storage de backup

Backups antigos (> 30 dias) sao raramente acessados. O Intelligent-Tiering move automaticamente para classes mais baratas.

Classe	Custo/GB/mes	Uso
S3 Standard	\$0.023	Backups recentes (< 30 dias)
S3 IA	\$0.0125	Backups antigos (30-90 dias)
S3 Glacier	\$0.004	Backups arquivados (> 90 dias)

Recomendacao 4: Escalar DR Nodes para 0 Fora de Horario Critico

Economia estimada: ~50% no custo de compute DR

O Warm Standby exige que a infra exista, mas os **nodes** podem ser escalados para 0 fora do horario comercial (18h-08h) e finais de semana. O control plane, NAT Gateway e RDS continuam ativos — ao escalar o node de volta, os pods sobem em poucos minutos.

Economia mensal estimada: ~\$18/mes (node t3.medium desligado ~60% do tempo)

Isso aumenta o RTO fora do horario comercial de ~1 min para ~8 min (tempo para node subir + pods inicializarem), o que e aceitavel para horarios de baixo volume.

Recomendacao 5: Karpenter para Autoscaling Inteligente

Economia estimada: ~20% no compute

Substituir o Cluster Autoscaler pelo Karpenter para provisionamento mais rapido e eficiente de nodes, escolhendo automaticamente o tipo de instancia mais barato disponivel.

5. Projecao de Custos (Producao Real vs Lab)

Cenario Producao Real (12 Meses - Sem Otimizacoes)

Periodo	Producao	DR Warm Standby	Total/Mes
Mes 1-6	\$160.00	\$148.00	\$308.00
Mes 7-12	\$180.00	\$148.00	\$328.00
Total Anual			\$3,816.00

Cenario Producao Real (12 Meses - Com Otimizacoes)

Periodo	Producao	DR (nodes off fora horario)	Total/Mes
Mes 1-6	\$120.00	\$130.00	\$250.00
Mes 7-12	\$135.00	\$130.00	\$265.00
Total Anual			\$3,090.00

Economia Total Estimada (Producao Real)

Metrica	Valor
Custo anual sem otimizacao	\$3,816.00
Custo anual otimizado	\$3,090.00
Economia anual	\$726.00 (19%)

Custo do Warm Standby vs Impacto de Downtime

Metrica	Valor
Custo mensal do DR Warm Standby	\$148.00
RTO com Cold DR	30-45 min
RTO com Warm Standby	3-5 min (teste: 58s)
Tempo de downtime evitado por incidente	~29-44 min
Perda por failover (estimativa honesta)	~R\$ 7.100 (~4 transacoes in-flight)
Perda evitada vs Cold DR (por incidente)	~R\$ 50.000+

Um unico incidente de queda de regio ja paga **5+ meses** de DR Warm Standby.

Cenario AWS Academy Learner Lab

Metrica	Valor
Budget disponivel	\$50.00
Custo/dia (producao + DR)	~\$10.25
Dias disponiveis	~4 dias
Uso planejado	4 dias (Prod + DR Warm Standby)
Custo estimado	~\$36.09
Margem restante	~\$13.91 (28%)

6. Dashboard de Custos

AWS Cost Explorer - Filtros Recomendados

Para acompanhamento financeiro, configurar os seguintes filtros no AWS Cost Explorer:

- 1. **Por Projeto:** Filtrar tag `Project = SolidaryTech`
- 2. **Por Serviço:** Filtrar tag `Service` para ver custo por microserviço
- 3. **Por Ambiente:** Filtrar tag `Environment` para comparar Production vs DR
- 4. **Por Centro de Custo:** Filtrar tag `CostCenter = NGO-Core`

Alertas de Budget (AWS Budgets)

Alerta	Threshold	Acao
Alerta de custo Lab	50% do budget (\$25)	Verificar se ja gravou o video
Alerta de custo Lab	80% do budget (\$40)	Destruir ambiente DR se ativo
Alerta critico Lab	90% do budget (\$45)	Executar <code>destroy.sh</code> imediatamente

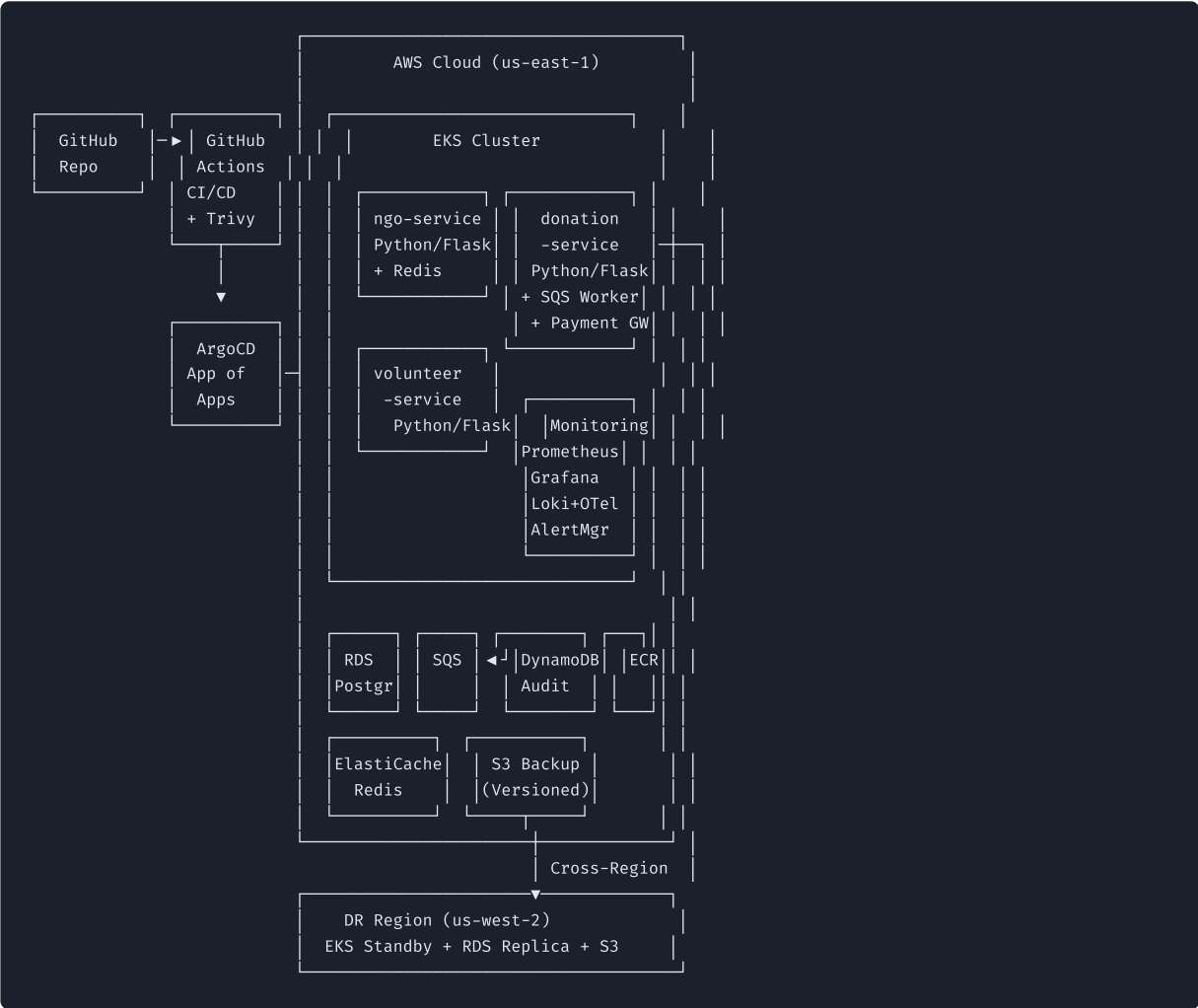
Producao real (recomendado):

Alerta	Threshold	Acao
Alerta de custo mensal	80% do budget (\$220)	Email para equipe DevOps
Alerta de custo mensal	100% do budget (\$280)	Email para gerencia
Alerta de forecast	Projecao > 120% do budget	Email para equipe + Slack
Alerta de anomalia	Custo diario > 2x da media	Email + Slack imediato

SolidaryTech - Hackathon Fase 5

Ecosistema de microsserviços para a plataforma **SolidaryTech**, uma empresa global sem fins lucrativos que conecta ONGs, doadores e voluntários, recebendo doações em diversas moedas (BRL, USD, EUR, GBP, JPY).

Arquitetura



Microsserviços

Serviço	Linguagem	Porta	Banco	Cache	Descrição
ngo-service	Python/Flask	8080	PostgreSQL	Redis	Cadastro e gestão de ONGs parceiras
donation-service	Python/Flask	8081	PostgreSQL + DynamoDB	Redis	Processamento de doações multi-moeda (Hot Path)
volunteer-service	Python/Flask	8082	PostgreSQL	-	Match entre voluntários e campanhas

Estrutura do Projeto (136 arquivos)

```
hackathon-solidarytech/  
├── microservices/  
│   └── ngo-service/          # Python - CRUD ONGs + Redis cache
```

```

├── donation-service/      # Python - Doações + SQS + Payment GW + DynamoDB
│   └── src/
│       ├── app.py         # API multi-moeda
│       ├── worker.py      # Consumer SQS
│       └── payment_gateway.py # Simulador de pagamento
├── volunteer-service/     # Python - Matching voluntários/campanhas
│   └── src/
│       ├── app.py         # API de voluntários e campanhas
│       └── models.py      # Volunteer, Campaign, Match
├── terraform/
│   ├── modules/           # VPC, EKS, RDS, SQS, S3, ECR, DynamoDB, ElastiCache
│   └── environments/      # production (us-east-1), dr (us-west-2)
├── kubernetes/
│   ├── base/              # Deployments, Services, HPA, PDB, NetworkPolicies
│   │   └── cluster/       # Metrics Server, Cluster Autoscaler
│   ├── monitoring/       # Prometheus, Grafana, Loki, OTel, AlertManager
│   └── argocd/            # App of Apps + Application CRDs
├── .github/workflows/     # 5 pipelines CI/CD (test, trivy, build, gitops, terraform)
├── docs/
│   ├── SLO-SLI-SLA.md    # Definições formais SRE
│   ├── PCN.md            # Plano de Continuidade de Negócios
│   ├── ITSM-lifecycle.md # Ciclo de vida de incidentes + AIOps
│   ├── finops-forecast.md # Custos, tagging, rightsizing, forecast
│   ├── pitch-executivo.md # Pitch de venda da solução
│   ├── evolucao-projeto.md # Evolução Fases 1-5
│   ├── decisoes-tecnicas.md # Decisões arquiteturais
│   └── roteiro-video.md  # Roteiro para gravação do vídeo
├── scripts/
│   ├── deploy.sh          # Deploy completo (Producao + DR Warm Standby)
│   ├── destroy.sh         # Destruir tudo (limpeza manual AWS CLI)
│   ├── stop-environment.sh # Pausar ambiente (escalar nodes para 0)
│   ├── start-environment.sh # Retomar ambiente (Prod + DR)
│   ├── dr-failover.sh     # Failover para região DR (RTO target: 3-5 min)
│   ├── dr-failback.sh     # Failback transparente DR → Prod (zero downtime)
│   ├── dr-data-sync.sh    # Sync bidirecional de dados (prod-DR, delta-only)
│   ├── setup-monitoring.sh # Instalar stack de monitoramento
│   ├── selfhealing-handler.sh # Remediacao automatica (AIOps)
│   ├── automated-rca.sh   # RCA automatizado + Post-Mortem
│   ├── dr-health-checker.sh # Health checker + failover automatico DR
│   ├── post-deploy-check.sh # Health check pos-deploy com auto-correcao
│   └── load-test.sh       # Teste de carga (seed, light, medium, heavy, ramp)

```

Quick Start

Teste Local (Docker Compose)

```

cd hackathon-solidarytech

# Subir stack local (PostgreSQL, Redis, LocalStack, 3 serviços, Prometheus, Grafana)
docker-compose up -d

# Criar filas SQS e tabela DynamoDB no LocalStack
./scripts/create-sqs-queue.sh

# Testar uma doação multi-moeda
curl -X POST http://localhost:8081/api/v1/donations \
  -H "Content-Type: application/json" \
  -d '{"donor_name":"John","amount":100,"currency":"USD","payment_method":"credit_card","ngo_id":1}'

# Ver moedas suportadas
curl http://localhost:8081/api/v1/donations/currencies

# Acessar Grafana
open http://localhost:3000 # admin/admin

```

Deploy na AWS Academy

```

# 1. Configurar credenciais (copiar do AWS Academy Learner Lab)
export AWS_ACCESS_KEY_ID="..."

```

```

export AWS_SECRET_ACCESS_KEY="..."
export AWS_SESSION_TOKEN="..."
export AWS_DEFAULT_REGION="us-east-1"

# 2. Deploy completo (Producao + DR Warm Standby em us-west-2)
./scripts/deploy.sh

# 3. Acessar dashboards
kubectl port-forward svc/grafana -n monitoring 3000:3000
kubectl port-forward svc/argocd-server 8443:443 -n argocd

# 4. Testar failover (RTO target: 3-5 min)
./scripts/dr-failover.sh

# 5. Failback transparente (zero downtime para o cliente)
./scripts/dr-failback.sh

```

Demo: Teste de Carga e Graficos no Grafana

Após o deploy, siga estes passos para popular o Grafana com dados reais e demonstrar auto-scaling.

Passo 1: Port-Forward dos servicos

```

# Abrir port-forward para os 3 microservicos + Grafana + Prometheus
kubectl port-forward svc/ngo-service -n solidarytech 8080:8080 &
kubectl port-forward svc/donation-service -n solidarytech 8081:8081 &
kubectl port-forward svc/volunteer-service -n solidarytech 8082:8082 &
kubectl port-forward svc/grafana -n monitoring 3000:3000 &
kubectl port-forward svc/prometheus -n monitoring 9091:9090 &

```

Passo 2: Criar dados iniciais (seed)

```

# Cria 5 ONGs, 10 voluntarios, 5 campanhas e 10 doacoes iniciais
./scripts/load-test.sh seed

```

Passo 3: Gerar carga e observar metricas no Grafana

```

# Abra o Grafana em http://localhost:3000 (admin/admin)
# Navegue ate o dashboard "SolidaryTech - Overview"
# Enquanto observa, execute um dos testes abaixo:

# Carga leve (5 req/s) - gera metricas basicas no Grafana
./scripts/load-test.sh light

# Carga media (20 req/s) - CPU sobe, HPA pode comecar a escalar
./scripts/load-test.sh medium

# Carga pesada (50 req/s) - HPA escala pods automaticamente
./scripts/load-test.sh heavy

# Rampa progressiva (5 -> 50 req/s) - demo completo de scaling
./scripts/load-test.sh ramp

# Demo completo: seed + rampa progressiva
./scripts/load-test.sh all

```

Passo 4: Verificar auto-scaling (HPA)

```

# Observar o HPA escalando pods em tempo real
watch kubectl get hpa -n solidarytech

# Exemplo de saida durante carga pesada:
# donation-service-hpa  cpu: 100%/60%  1 8 4  ← escalou de 1 para 4 replicas!
# ngo-service-hpa      cpu: 17%/70%   1 4 1
# volunteer-service-hpa cpu: 14%/70%   1 4 1

```

Passo 5: Escalar nodes (se necessario)

```
# No AWS Academy, o Cluster Autoscaler nao opera (limitacao IAM).
# Para escalar nodes manualmente:
aws eks update-nodegroup-config \
  --cluster-name solidaritytech-eks-production \
  --nodegroup-name solidaritytech-node-group \
  --scaling-config minSize=2,maxSize=5,desiredSize=3
```

Passo 6: Verificar estado geral

```
# Health check completo com auto-correcao
./scripts/post-deploy-check.sh

# Status rapido (pods, HPA, nodes)
./scripts/load-test.sh status
```

Dashboards Disponiveis no Grafana

Dashboard	O que mostra
Platform Overview	Request rate, latencia p99, erros, pods, HPA, CPU/memoria + Regiao Ativa
Business Metrics - Doacoes	Doacoes processadas, total arrecadado, distribuicao por moeda + Regiao Ativa
SRE - Golden Metrics (Donation Service)	Latencia, trafego, erros, saturacao do Hot Path + Regiao Ativa
SLO/SLI Tracking - Donation Service	Error budget, SLO compliance, burn rate + Regiao Ativa
Infrastructure & Cluster	Nodes, pods, HPA, CPU/memoria por namespace + Regiao Ativa
Disaster Recovery - Status & Impacto	RTO/RPO, integridade de dados, perda estimada, timeline + Regiao Ativa

Demo de DR Failover e Failback

```
# Modo dry-run (simula sem executar)
./scripts/dr-health-checker.sh --dry-run

# Failover manual (RTO target: 3-5 min)
./scripts/dr-failover.sh

# Sync bidirecional de dados (prod-DR)
./scripts/dr-data-sync.sh bidirectional

# Failback transparente (zero downtime para o cliente)
./scripts/dr-failback.sh

# Verificar regiao ativa via API
curl http://localhost:8081/region

# Failover automatico real (requer DR deployado em us-west-2)
./scripts/dr-health-checker.sh --once --auto-failover
```

Limitacoes do AWS Academy

O AWS Academy Learner Lab impoe restricoes de IAM que afetam algumas funcionalidades:

Funcionalidade	Restricao	Mitigacao
Cluster Autoscaler	LabEksNodeRole sem <code>autoscaling:DescribeAutoScalingGroups</code>	Node scaling via CLI (<code>aws eks update-nodegroup-config</code>)
SQS Worker	LabEksNodeRole sem <code>sqs:ReceiveMessage</code>	Worker escalado a 0; doacoes processadas sincronamente
RDS Read Replica cross-region	Sem <code>rds:CreateDBInstanceReadReplica</code>	DR usa RDS standalone; em producao seria Read Replica com RPO ~1s
EKS Managed Node Group SG	Pods usam SG auto-criado pelo EKS, diferente do Terraform	Modulo RDS aceita ambos os SGs (node + cluster)
IMDS hop limit	Padrao = 1, pods nao acessam credenciais	Corrigido via <code>modify-instance-metadata-options</code>

Em producao real com IAM completo, todas as funcionalidades operam nativamente (Read Replica cross-region, Cluster Autoscaler, SQS Worker). Documentacao detalhada: [docs/decisoes-tecnicas.md](#) [secao 10](#).

Gerenciamento de Custos

```
# Pausar ambiente (escala nodes para 0, mantém control planes e RDS replica)
./scripts/stop-environment.sh

# Retomar ambiente (Produção + DR Warm Standby)
./scripts/start-environment.sh

# Destruir TUDO (limpeza manual via AWS CLI - não depende de Terraform)
./scripts/destroy.sh
```

Cobertura dos 5 Eixos de Avaliação

0. Fundação DevOps (Fases 1-4) - Obrigatório

Requisito	Implementação	Evidência
Docker	Dockerfiles multi-stage otimizados (Python)	<code>microservices/*/Dockerfile</code>
Kubernetes (EKS)	Deployments com HPA, probes, PDB, resource limits	<code>kubernetes/base/</code>
IaC (Terraform)	8 módulos: VPC, EKS, RDS, SQS, S3, ECR, DynamoDB, ElastiCache	<code>terraform/modules/</code>
CI/CD (GitHub Actions)	5 pipelines com testes, Trivy SAST/SCA, build/push ECR	<code>.github/workflows/</code>
GitOps (ArgoCD)	App of Apps com auto-sync e self-heal	<code>kubernetes/argocd/</code>
Observabilidade	Prometheus + Grafana + Loki + OTel + AlertManager	<code>kubernetes/monitoring/</code>
Segurança	NetworkPolicies (deny-all), SecurityContext hardened (non-root, drop ALL, readOnly), ECR IMMUTABLE, deletion_protection, automountServiceAccountToken:false	<code>kubernetes/base/network-policies.yaml</code>

1. SRE: Golden Metrics e SLOs

- **SLI Latência:** p99 do donation-service < 500ms (SLO: 99.9%)
- **SLI Disponibilidade:** Taxa de erro < 0.1% (SLO: 99.9%)
- **Error Budget:** 43.2 min/mês - política de consumo documentada
- **MTTR:** Reduzido via selfhealing automático e RCA automatizado
- **Dashboard SRE:** 2 dashboards Grafana (Golden Metrics + SLO/Error Budget)
- **Alerting Rules:** 8 regras para detecção proativa

- Documentação: [docs/SLO-SLI-SLA.md](#)

2. FinOps: Otimização Financeira

- **Tagging:** TODOS os recursos com Project, Environment, CostCenter, ManagedBy
- **Rightsizing:** Requests/limits calibrados (donation-service 2x por ser Hot Path)
- **Forecast:** ~\$352/mês (produção + DR) com 5 recomendações de otimização
- **Scripts de economia:** stop/start para pausar quando não usar
- Documentação: [docs/finops-forecast.md](#)

3. ITSM e AIOps

- **SelfHealing:** PDB + AlertManager + webhook de remediação automática (Python)
- **Auto-Scaling 3 camadas:**
- **Pod:** HPA com CPU + memória — donation-service (Hot Path) escala até 8 réplicas, scale-up de 4 pods/30s
- **Node:** Cluster Autoscaler v1.32.0 + Metrics Server v0.7.2 — escala nodes automaticamente de 1 a 5
- **DR:** Health checker automático com failover em ~5 min após 3 falhas consecutivas
- **RCA Automatizado:** Script que coleta logs, eventos, métricas e gera relatório
- **Ciclo de Incidentes:** 6 fases (Detecção → Post-Mortem)
- **Post-Mortem Blameless:** Template estruturado
- Documentação: [docs/ITSM-lifecycle.md](#)

4. Multicloud, Segurança e DR

- **PCN:** RTO 3-5 minutos (teste em lab: 58s) / RPO 0 para dados commitados (sync bidirecional)
- **DR Warm Standby:** Workload mínimo **sempre ativo** em us-west-2 (1 node + RDS + apps com 1 replica)
- **Failover com sync:** `dr-failover.sh` sincroniza dados prod→DR antes do failover, garantindo integridade
- **Failback transparente:** `dr-failback.sh` com 8 etapas, dual-active transitorio, zero downtime para o cliente
- **Sync bidirecional:** `dr-data-sync.sh` com delta-only transfer, dedup por transaction_id/cnpj/email
- **Perda honesta no failover:** ~4 transacoes in-flight durante 58s de RTO (~R\$ 7.100 de impacto)
- **Identificacao de regioao:** Grafana "Regiao Ativa" (PROD/DR) em todos os 6 dashboards + endpoint `/region` nos 3 microsservicos + Prometheus `external_labels`
- **Failover Automático:** `dr-health-checker.sh --daemon --auto-failover` monitora produção e dispara failover automaticamente após 3 falhas consecutivas
- **Segurança Hardened (4 camadas):** credenciais mascaradas em scripts, ECR IMMUTABLE, deletion_protection no RDS, securityContext non-root com drop ALL em todos os pods, NetworkPolicies deny-all + regras explicitas, automountServiceAccountToken:false, validacao de entrada (paginacao max 100, valores monetarios), Grafana admin via secretKeyRef
- Documentação: [docs/PCN.md](#) , [docs/deciso-es-tecnicas.md](#)

Tecnologias Utilizadas

Categoria	Tecnologia
Linguagens	Python 3.13
Frameworks	Flask, SQLAlchemy, Gunicorn
Bancos de Dados	PostgreSQL 15, DynamoDB, Redis
Container	Docker (multi-stage builds)
Orquestração	Kubernetes (EKS 1.36)
IaC	Terraform
CI/CD	GitHub Actions
GitOps	ArgoCD (App of Apps)
Observabilidade	Prometheus, Grafana, Loki, OpenTelemetry, AlertManager
Mensageria	Amazon SQS
Segurança	Trivy, NetworkPolicies, SecurityContext
DR	Warm Standby (us-west-2), sync bidirecional, RTO 3-5 min (teste: 58s), failback transparente

Equipe

Consulte o documento de entrega do projeto (PDF) para a lista completa de integrantes.